

Hub de

Apostilas, exerci

{ }

JSON

GET/POST

HTTP

Desenvolvimento

REST, Next.j

Desenvolvimento de APIs para a Web

REST, Next.js, Prisma e segurança

Hub de Estudos

Apostilas, exercícios e simulados

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia.

Desenvolvido por TecPro Tecnologias.

Sumário

Capítulo 1 – APIs e Web Services

Quando você abre o aplicativo do banco e vê o seu saldo, quando um site de compras calcula o frete a partir do CEP ou quando um sistema emite uma nota fiscal eletrônica, há sempre dois programas conversando: um que pede uma informação e outro que a entrega. Essa conversa acontece por meio de uma **API**. Neste capítulo você vai entender o que é uma API, como funciona o protocolo HTTP, que transporta essas conversas na web, e quais são os principais estilos de serviço web: SOAP, REST e GraphQL. No final, vai consultar uma API pública de verdade e conhecer o projeto que será construído ao longo da apostila.

1.1 O que é uma API

API é a sigla de *Application Programming Interface*, ou interface de programação de aplicações. É um conjunto de regras que define como um programa pode pedir algo a outro: quais pedidos existem, que dados cada pedido precisa receber e o que vem de volta.

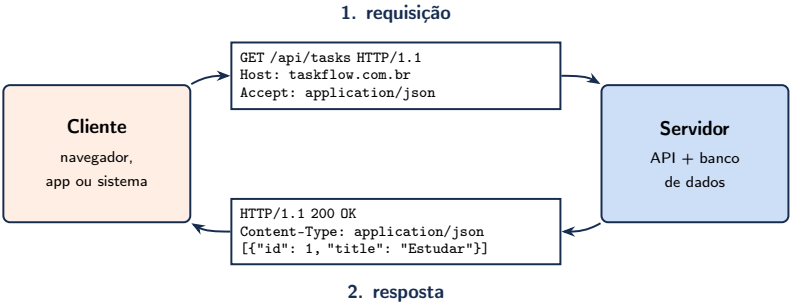
Uma comparação comum é a do restaurante. O cliente não entra na cozinha: ele consulta o cardápio e faz o pedido ao garçom, que leva o pedido à cozinha e traz o prato. O cardápio é o **contrato** da API (o que pode ser pedido), o garçom é a API, e a cozinha é o sistema que faz o trabalho. O cliente não precisa saber como a cozinha funciona, e a cozinha pode mudar de fogão sem que o cliente perceba, desde que o cardápio continue o mesmo.

Esse é o grande valor de uma API: ela **separa quem usa de quem implementa**. Um mesmo servidor pode atender, ao mesmo tempo, o site, o aplicativo de celular e o sistema de um parceiro, todos usando o mesmo contrato.

O termo API é amplo. As funções de uma biblioteca, como o `readFile` do Node.js, também formam uma API. Nesta apostila, porém, o foco são as **APIs web**: aquelas acessadas pela rede, usando o protocolo HTTP. Quando essas APIs são oferecidas por um sistema para ser usadas por outros sistemas, também são chamadas de **serviços web** (*web services*).

1.2 Cliente, servidor e o protocolo HTTP

Toda conversa com uma API web segue o modelo **cliente-servidor**: o **cliente** envia uma **requisição** (*request*) e o **servidor** devolve uma **resposta** (*response*). O cliente pode ser um navegador, um aplicativo de celular ou outro servidor. As regras dessa troca são as do **HTTP** (*HyperText Transfer Protocol*), o mesmo protocolo que o navegador usa para carregar páginas.



o servidor não guarda memória entre uma requisição e outra (sem estado)

Uma requisição HTTP tem quatro partes:

- o **método**, que diz o que o cliente quer fazer: `GET` para buscar, `POST` para criar, `PUT` e `PATCH` para alterar, `DELETE` para excluir;
- a **URL**, que diz em que recurso a ação será feita, como `/api/tasks`;
- os **cabeçalhos** (*headers*), pares nome e valor com informações extras, como o formato aceito na resposta;
- o **corpo** (*body*), opcional, com os dados enviados, por exemplo os campos de uma tarefa nova.

A resposta tem três: o **código de status**, um número que resume o resultado (`200` deu certo, `404` não encontrado, `500` erro no servidor), os **cabeçalhos** e o **corpo**, com os dados pedidos. Por baixo de tudo, requisição e resposta são apenas texto. Veja como fica a criação de uma tarefa:

```
POST /api/tasks HTTP/1.1
Host: taskflow.com.br
Content-Type: application/json

{"title": "Estudar HTTP", "priority": "HIGH"}
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/tasks/42

{"id": 42, "title": "Estudar HTTP", "priority": "HIGH"}
```

A linha em branco separa os cabeçalhos do corpo. O cabeçalho `Content-Type` informa o formato do corpo; aqui, `application/json`. O status `201 Created` indica que um recurso novo foi criado, e o cabeçalho `Location` diz onde ele pode ser encontrado. Os métodos e os códigos de status são estudados em detalhe nos Capítulos 2 e 8.

Uma característica essencial do HTTP é ser **sem estado** (*stateless*): cada requisição é independente, e o servidor não se lembra das anteriores. Se o servidor precisa saber quem está pedindo, a própria requisição tem de trazer essa informação, normalmente

um **token** em um cabeçalho. É assim que funciona o login em APIs, assunto do Capítulo 9.

1.3 A anatomia de uma URL

A URL (*Uniform Resource Locator*) identifica onde está o recurso. Cada parte tem uma função:

```
https://api.taskflow.com.br:443/v1/tasks?status=DONE&page=2
```

Parte	Exemplo	Função
protocolo	`https`	como a conversa acontece; o `s` indica conexão criptografada
domínio	`api.taskflow.com.br`	o nome do servidor
porta	`443`	a "porta de entrada"; 443 é o padrão do HTTPS e costuma ser omitida
caminho	`/v1/tasks`	o recurso dentro do servidor
query string	`?status=DONE&page=2`	parâmetros extras, como filtros e paginação, em pares nome=valor separados por `&`

Na *query string*, espaços e acentos não podem aparecer como são: eles são codificados, e "São Paulo" vira `S%C3%A3o%20Paulo`. O `fetch` e as bibliotecas de HTTP cuidam disso com `encodeURIComponent` ou com a classe `URLSearchParams`.

1.4 Serviços web com SOAP

Os primeiros serviços web populares, no início dos anos 2000, usavam o **SOAP** (*Simple Object Access Protocol*), um protocolo padronizado pelo W3C. No SOAP, toda mensagem é um documento **XML** com uma estrutura fixa, o **envelope**, e o serviço é descrito em um arquivo **WSDL** (*Web Services Description Language*), também em XML, que lista as operações, os parâmetros e os tipos de cada uma. A partir do WSDL, ferramentas geram automaticamente o código do cliente.

```
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
    xmlns:t="http://taskflow.com.br/ws">
  <soap:Header/>
  <soap:Body>
    <t:BuscarTarefa>
      <t:id>42</t:id>
    </t:BuscarTarefa>
  </soap:Body>
</soap:Envelope>
```

Repare que a operação, `BuscarTarefa`, aparece dentro da mensagem. No SOAP, todas as chamadas costumam ir para uma mesma URL, quase sempre com o método `POST`, e é o conteúdo do envelope que diz o que fazer. O padrão também define como devolver

erros (o elemento `Fault`) e tem extensões para segurança e transações, como o WS-Security.

O SOAP é verboso e rígido, mas essa rigidez é justamente o que alguns sistemas querem: um contrato formal, validado automaticamente. Por isso ele continua presente em bancos, seguradoras e em sistemas de governo. No Brasil, a emissão da Nota Fiscal Eletrônica (NF-e) é feita por serviços web SOAP das secretarias da Fazenda.

1.5 APIs REST

O **REST** (*Representational State Transfer*) não é um protocolo, e sim um **estilo de arquitetura**, descrito por Roy Fielding em sua tese de doutorado, em 2000. A ideia central é organizar a API em torno de **recursos**, as "coisas" do sistema (tarefas, usuários, pedidos), cada um identificado por uma URL, e usar os **métodos do próprio HTTP** para dizer o que fazer com eles:

```
GET    /api/tasks       lista as tarefas
POST   /api/tasks       cria uma tarefa
GET    /api/tasks/42    busca a tarefa 42
PATCH  /api/tasks/42    altera parte da tarefa 42
DELETE /api/tasks/42    exclui a tarefa 42
```

O servidor devolve uma **representação** do recurso, hoje quase sempre em **JSON**, um formato de texto leve e fácil de ler tanto por pessoas quanto por programas:

```
{
  "id": 42,
  "title": "Estudar HTTP",
  "status": "PENDING",
  "priority": "HIGH",
  "dueDate": "2026-11-20"
}
```

Compare com o SOAP: em vez de uma operação `BuscarTarefa` escrita dentro de um envelope, o REST usa `GET` na URL da tarefa. As regras do estilo, como a ausência de estado e o uso de URLs para identificar recursos, são o tema do Capítulo 2. Por ser simples, usar o HTTP como ele foi pensado e funcionar bem com JavaScript, o REST se tornou o estilo dominante nas APIs web, e é o estilo usado nesta apostila.

1.6 GraphQL

O **GraphQL** foi criado pelo Facebook em 2012 e aberto ao público em 2015. Ele é uma **linguagem de consulta** para APIs: em vez de várias URLs, há um único endereço, normalmente `/graphql`, e o cliente envia uma consulta dizendo exatamente quais campos quer receber.

```
query {
  usuario(id: 7) {
    nome
    tarefas(status: PENDING) {
```

```
      title
      dueDate
    }
  }
}
```

```
{
  "data": {
    "usuario": {
      "nome": "Ana",
      "tarefas": [
        { "title": "Estudar HTTP", "dueDate": "2026-11-20" }
      ]
    }
  }
}
```

A resposta tem o mesmo formato da consulta. Isso resolve dois problemas comuns do REST: receber campos demais, que o cliente não vai usar (*over-fetching*), e precisar de várias requisições para montar uma tela, como uma para o usuário e outra para as tarefas dele (*under-fetching*). Em troca, o servidor fica mais complexo, e recursos simples do HTTP, como o *cache* por URL, deixam de funcionar sozinhos. O GraphQL brilha em aplicações com telas muito variadas, como redes sociais; para a maioria das APIs, o REST continua mais simples.

1.7 Outros estilos de comunicação

Três outras formas de comunicação aparecem com frequência ao lado das APIs:

- **gRPC**: criado pelo Google, usa o HTTP/2 e um formato binário, o *Protocol Buffers*, muito mais compacto que o JSON. É comum na comunicação entre servidores de uma mesma empresa, em que o desempenho importa mais que a facilidade de leitura.
- **WebSocket**: abre uma conexão que fica aberta, e o servidor pode enviar dados ao cliente a qualquer momento, sem esperar uma requisição. É a base de chats, placares ao vivo e notificações em tempo real.
- **Webhook**: é o caminho inverso de uma API. Em vez de o seu sistema perguntar de tempos em tempos "o pagamento foi aprovado?", o serviço de pagamento faz uma requisição `POST` para uma URL do seu sistema assim que o pagamento é aprovado.

1.8 Comparando SOAP, REST e GraphQL

	SOAP	REST	GraphQL
o que é	protocolo	estilo de arquitetura	linguagem de consulta
formato	XML (envelope)	qualquer um; quase sempre JSON	JSON
endereços	uma URL por serviço	uma URL por recurso	uma URL só
operação indicada	no envelope	pelo método HTTP	na consulta
contrato	WSDL, obrigatório	opcional (OpenAPI, Capítulo 10)	esquema, obrigatório
uso típico	bancos, governo, sistemas legados	APIs públicas e a maioria dos sistemas web	telas com dados muito variados

Nenhum estilo é melhor em tudo. A escolha depende de quem vai usar a API, do tipo de dado e das exigências do sistema. Um mesmo produto pode, inclusive, usar mais de um: REST para o aplicativo, SOAP para falar com o sistema fiscal e WebSocket para as notificações.

1.9 Consultando uma API pública

A melhor forma de entender uma API é usar uma. O **ViaCEP** é um serviço gratuito que devolve o endereço de um CEP e não exige cadastro. Como a consulta usa o método `GET`, basta colar a URL no navegador:

```
https://viacep.com.br/ws/01001000/json/
```

Pelo terminal, a ferramenta `curl` faz a mesma requisição e mostra a resposta. Ela já vem instalada no Windows 10 e 11, no macOS e no Linux; no PowerShell, digite `curl.exe`. A opção `-i` mostra também a linha de status e os cabeçalhos:

```
$ curl -i https://viacep.com.br/ws/01001000/json/
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "complemento": "lado ímpar",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP",
  "ddd": "11"
}
```

A resposta real tem mais campos e cabeçalhos; aqui aparecem só os principais. Em um programa, a requisição é feita com o `fetch`, que existe no navegador e, desde a versão 18, também no Node.js:

```
// cep.mjs  -- rode com: node cep.mjs
const cep = "01001000";
const url = `https://viacep.com.br/ws/${cep}/json/`;

const resposta = await fetch(url);
console.log(resposta.status);     // 200

const endereco = await resposta.json();
console.log(`${endereco.logradouro}, ${endereco.localidade}`);
// Praça da Sé, São Paulo
```

O `fetch` devolve uma *Promise* com a resposta; `resposta.status` é o código de status, e `resposta.json()` lê o corpo e o converte em um objeto JavaScript. Troque `json` por `xml` no fim da URL e o mesmo serviço responde em XML, assunto do Capítulo 5.

Experimente também um CEP que não existe, como `99999999`. O ViaCEP responde com status `200` e o corpo `{"erro": "true"}`. É uma escolha de projeto discutível: o Capítulo 8 mostra por que, em uma API REST, o mais adequado seria o status `404`.

1.10 O projeto da apostila: a API do TaskFlow

Ao longo da apostila, você vai construir a API do **TaskFlow**, um gerenciador de tarefas em que cada pessoa cria uma conta e organiza as próprias tarefas. A cada capítulo, a API ganha uma parte nova, até ser publicada na internet com domínio próprio. As ferramentas são as mesmas usadas por empresas no dia a dia:

- **Next.js** com **TypeScript**, para criar as rotas da API (Capítulos 3 e 4);
- **PostgreSQL** na nuvem, com o NeonDB, e o **Prisma** para acessar o banco (Capítulos 3 e 7);
- **Zod**, para validar os dados recebidos (Capítulo 6);
- **JWT**, para a autenticação (Capítulo 9);
- **Vitest**, para os testes automatizados (Capítulo 11);
- **Vercel**, para a publicação (Capítulo 12).

Ao final, a API terá estes *endpoints*, ou seja, as combinações de método e URL que ela atende:

Método	URL	O que faz
`POST`	`/api/auth/register`	cria uma conta
`POST`	`/api/auth/login`	entra e recebe um token
`GET`	`/api/tasks`	lista as tarefas de quem está logado, com filtros
`POST`	`/api/tasks`	cria uma tarefa
`GET`	`/api/tasks/:id`	busca uma tarefa
`PATCH`	`/api/tasks/:id`	altera uma tarefa
`DELETE`	`/api/tasks/:id`	exclui uma tarefa

O `:id` indica uma parte variável da URL: `/api/tasks/42` é a tarefa 42. Para acompanhar, você vai precisar do Node.js (versão 20 ou mais nova), do VS Code e de uma conta gratuita no GitHub, que também serve para entrar no NeonDB e na Vercel.

Erros Comuns

- **Achar que API é sinônimo de REST.** REST é um estilo; SOAP, GraphQL e gRPC também são formas de construir APIs.
- **Achar que o servidor "lembra" do cliente.** O HTTP é sem estado: cada requisição precisa trazer tudo o que o servidor precisa saber, inclusive quem está pedindo.
- **Confundir a URL com o *endpoint*.** O mesmo caminho, `/api/tasks`, é um *endpoint* com `GET` (listar) e outro com `POST` (criar).
- **Esquecer o `await` no `fetch`.** Sem ele, a variável guarda uma *Promise*, e não a resposta. O mesmo vale para `resposta.json()`.
- **Confiar apenas no status.** Algumas APIs, como o ViaCEP com um CEP inexistente, respondem `200` e indicam o erro no corpo. Leia a documentação de cada API.
- **Colocar acentos e espaços direto na *query string*.** Use `encodeURIComponent` ou `URLSearchParams` para montar os parâmetros.

Capítulo 2 – Design RESTful e métodos HTTP

No Capítulo 1, o REST apareceu como o estilo que organiza a API em torno de recursos e usa os métodos do HTTP. Mas dizer que uma API "é REST" não basta: duas APIs REST podem ser fáceis ou difíceis de usar dependendo de como foram projetadas. Neste capítulo você vai conhecer as regras do estilo REST, aprender a escolher boas URLs, entender o significado preciso de cada método HTTP, inclusive os conceitos de método seguro e idempotente, e ver as convenções de filtros, paginação e versionamento. No final, vai projetar no papel a API completa do TaskFlow, antes de escrever qualquer código.

2.1 Recursos e representações

Em REST, tudo gira em torno de **recursos**: as "coisas" que o sistema guarda e que fazem sentido para quem usa a API. No TaskFlow, os recursos são os usuários e as tarefas. Em uma loja, seriam produtos, clientes e pedidos.

Cada recurso tem um identificador único, a sua **URI** (*Uniform Resource Identifier*), como `/api/tasks/42`. A URI é o "endereço" do recurso, e não muda conforme a ação: buscar, alterar e excluir a tarefa 42 usam todas a mesma URI, com métodos diferentes.

O cliente nunca recebe o recurso em si, que está guardado no banco de dados, e sim uma **representação** dele: um retrato em algum formato, quase sempre JSON. O mesmo recurso pode ter várias representações. O cliente diz qual prefere com o cabeçalho `Accept`, e o servidor informa qual enviou com o `Content-Type`:

```
GET /api/tasks/42 HTTP/1.1
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"id": 42, "title": "Estudar REST", "status": "PENDING"}
```

Essa escolha do formato pelos cabeçalhos se chama **negociação de conteúdo** e volta no Capítulo 5, com JSON e XML.

2.2 As restrições do REST

Roy Fielding definiu o REST como um conjunto de **restrições**: regras que, quando seguidas, dão ao sistema escalabilidade, simplicidade e independência entre as partes. Uma API é RESTful quando respeita essas regras:

Restrição	O que significa
cliente-servidor	interface e dados ficam separados; o site, o aplicativo e o servidor evoluem de forma independente
sem estado	cada requisição traz tudo o que o servidor precisa; ele não guarda sessão entre uma e outra
cache	a resposta diz se pode ser guardada e reaproveitada, o que poupa requisições
interface uniforme	recursos identificados por URIs, manipulados por representações, com mensagens que se explicam sozinhas
sistema em camadas	o cliente não sabe se fala direto com o servidor ou com intermediários, como um balanceador de carga ou uma CDN
código sob demanda (opcional)	o servidor pode enviar código para o cliente executar, como um *script*

A restrição **sem estado** é a que mais influencia o projeto. Como o servidor não guarda nada entre as requisições, qualquer servidor de um grupo pode atender qualquer requisição. É isso que permite colocar dez cópias da API atrás de um balanceador de carga quando o número de usuários cresce. O preço é que o cliente precisa mandar, a cada requisição, a sua identificação, normalmente um *token*.

2.3 Boas URLs

A URL é a primeira coisa que o desenvolvedor vê da sua API, e um bom padrão torna a API previsível: quem aprendeu a listar tarefas consegue adivinhar como listar usuários. As convenções mais aceitas são estas:

- **Use substantivos, nunca verbos.** A ação já está no método HTTP. `POST /tasks`, e não `POST /criarTarefa`.
- **Use o plural para coleções.** `/tasks` é a coleção; `/tasks/42` é um item dela. O mesmo nome serve aos dois casos.
- **Mostre a hierarquia quando ela existe.** As tarefas do usuário 7 ficam em `/users/7/tasks`. Evite passar de dois níveis: `/users/7/tasks/42/comments/3` já é difícil de manter.
- **Use letras minúsculas e hífens.** `/task-categories`, e não `/TaskCategories` nem `/task_categories`. Partes da URL podem diferenciar maiúsculas de minúsculas, o que confunde.
- **Filtros vão na *query string*.** `/tasks?status=DONE`, e não `/tasks/done`: a tarefa concluída continua sendo uma tarefa.
- **Não exponha detalhes de implementação.** Nada de `/tasks.php` ou `/api/tabela_tarefas`. Se a tecnologia mudar, a URL continua a mesma.

Evite	Prefira
`GET /getTasks`	`GET /tasks`
`POST /tasks/create`	`POST /tasks`
`GET /task/42`	`GET /tasks/42`
`POST /tasks/42/delete`	`DELETE /tasks/42`
`GET /tasks/pendentes`	`GET /tasks?status=PENDING`
`GET /tasksDoUsuario?id=7`	`GET /users/7/tasks`

Há casos em que uma ação não se encaixa bem em criar, ler, alterar ou excluir, como "reenviar o e-mail de confirmação". A saída usual é tratar a ação como um subrecurso e usar `POST`: `POST /users/7/confirmation-email`. Use isso com moderação; na maioria das vezes, a ação pode ser expressa como alteração de um campo, como `PATCH /tasks/42` com `{"status": "DONE"}` em vez de `POST /tasks/42/concluir`.

2.4 Os métodos HTTP

O método diz **o que** o cliente quer fazer com o recurso indicado pela URL. Cada método tem um significado definido pela especificação do HTTP (a RFC 9110), e respeitá-lo é o que permite a navegadores, *caches* e bibliotecas tratarem a sua API corretamente.

Método	Uso	Corpo	Seguro	Idempotente
`GET`	buscar um recurso ou uma coleção	não	sim	sim
`POST`	criar um recurso em uma coleção	sim	não	não
`PUT`	substituir o recurso inteiro	sim	não	sim
`PATCH`	alterar parte do recurso	sim	não	não*
`DELETE`	excluir o recurso	não	não	sim
`HEAD`	como o GET, mas só os cabeçalhos	não	sim	sim
`OPTIONS`	perguntar quais métodos a URL aceita	não	sim	sim

* O PATCH pode ser idempotente, dependendo de como a alteração é descrita (Seção 2.6).

O `HEAD` serve, por exemplo, para saber o tamanho de um arquivo pelo cabeçalho `Content-Length` sem baixá-lo. O `OPTIONS` é usado automaticamente pelo navegador antes de certas requisições entre domínios diferentes, no mecanismo chamado CORS, estudado no Capítulo 12.

2.5 Métodos seguros e idempotentes

Duas propriedades da tabela anterior merecem atenção, porque são cobradas com frequência e evitam erros graves.

Um método é **seguro** quando não altera nada no servidor: ele só lê. `GET`, `HEAD` e `OPTIONS` são seguros. Por isso, navegadores e robôs de busca seguem links (que fazem `GET`) livremente, e ferramentas de *cache* guardam as respostas. Uma API que exclui algo em um `GET`, como `GET /tasks/42/delete`, corre o risco de ver os dados sumirem quando um robô ou uma pré-visualização de link visitar a URL.

Um método é **idempotente** quando repetir a mesma requisição várias vezes deixa o servidor no mesmo estado que fazê-la uma vez só. Pense em um elevador: apertar o botão do 5º andar uma ou dez vezes leva ao mesmo lugar.

- `DELETE /tasks/42` é idempotente: depois da primeira vez, a tarefa já não existe, e as repetições não mudam nada. A resposta pode mudar (a segunda recebe `404`), mas o **estado** do servidor é o mesmo.
- `PUT /tasks/42` com a tarefa completa é idempotente: gravar os mesmos dados de novo dá no mesmo.

- `POST /tasks` **não** é idempotente: cada repetição cria uma tarefa nova.

Isso importa na prática por causa das falhas de rede. Se o cliente envia uma requisição e a conexão cai antes da resposta, ele não sabe se o servidor a recebeu. Com um método idempotente, basta repetir. Com um `POST`, repetir pode criar um pedido em dobro, e é por isso que sites de compra pedem para você não apertar o botão "Pagar" duas vezes. APIs de pagamento resolvem o problema com uma **chave de idempotência**: o cliente gera um código único e o envia em um cabeçalho, como `Idempotency-Key`; se o servidor já processou aquele código, devolve o resultado anterior em vez de cobrar de novo.

2.6 PUT ou PATCH?

Os dois alteram um recurso existente, mas de formas diferentes. O `PUT` **substitui** o recurso inteiro pela representação enviada; o `PATCH` aplica uma **alteração parcial**. Considere a tarefa 42:

```
{"id": 42, "title": "Estudar REST",
 "status": "PENDING", "priority": "HIGH"}
```

Para marcá-la como concluída, o `PATCH` envia só o que muda:

```
PATCH /api/tasks/42
{"status": "DONE"}
```

Com `PUT`, é preciso enviar a tarefa completa. Se o cliente mandar só `{"status": "DONE"}`, um servidor que segue a regra à risca entende que o título e a prioridade foram retirados:

```
PUT /api/tasks/42
{"title": "Estudar REST", "status": "DONE", "priority": "HIGH"}
```

Na prática, o `PATCH` é o mais usado para edições, porque os formulários costumam alterar poucos campos e a requisição fica menor. Um `PATCH` como o acima, que define valores, é idempotente. Já um `PATCH` que descreve uma operação relativa, como "some 1 ao contador de visualizações", não é: cada repetição soma de novo. Por isso a especificação não garante a idempotência do `PATCH`. A API do TaskFlow usa `PATCH` para editar tarefas.

2.7 Os códigos de status mais usados

O código de status resume o resultado em três dígitos, e o primeiro indica a família:

Família	Significado
`1xx`	informação: a requisição foi recebida e continua em processamento (raro em APIs)
`2xx`	sucesso
`3xx`	redirecionamento: o recurso está em outro lugar
`4xx`	erro do cliente: a requisição tem algum problema
`5xx`	erro do servidor: a requisição estava certa, mas o servidor falhou

A distinção entre `4xx` e `5xx` é importante: com `4xx`, o cliente precisa corrigir algo antes de tentar de novo; com `5xx`, a mesma requisição pode funcionar mais tarde. Os códigos que aparecem em quase toda API são estes, e o Capítulo 8 trata de como escolher e montar as respostas de erro:

Código	Quando usar
`200 OK`	sucesso com corpo na resposta: GET, PUT, PATCH
`201 Created`	POST que criou um recurso; envie o cabeçalho `Location` com a URI dele
`204 No Content`	sucesso sem corpo, típico do DELETE
`400 Bad Request`	dados malformados ou inválidos
`401 Unauthorized`	falta autenticação, ou o *token* é inválido
`403 Forbidden`	autenticado, mas sem permissão para aquele recurso
`404 Not Found`	o recurso não existe
`405 Method Not Allowed`	a URL existe, mas não aceita aquele método
`409 Conflict`	conflito com o estado atual, como um e-mail já cadastrado
`500 Internal Server Error`	falha inesperada no servidor

2.8 Filtros, ordenação e paginação

Uma coleção pode ter milhares de itens, e devolver tudo de uma vez deixa a API lenta e a resposta enorme. As operações sobre coleções usam a *query string*, com nomes que sigam sempre o mesmo padrão:

```
GET /api/tasks?status=PENDING&priority=HIGH
GET /api/tasks?search=relatorio
GET /api/tasks?sort=dueDate
GET /api/tasks?sort=-createdAt
GET /api/tasks?page=2&limit=10
```

Os filtros usam o nome do próprio campo. A busca por texto costuma usar `search` ou `q`. Na ordenação, uma convenção comum é o sinal de menos indicar a ordem decrescente: `-createdAt` lista as mais recentes primeiro. Na **paginação**, `page` indica a página e `limit` quantos itens vêm em cada uma. A resposta deve informar ao cliente quanto falta, para que ele saiba se mostra o botão "próxima página":

```
{
  "data": [
    {"id": 11, "title": "Revisar capítulo 1"},
    {"id": 12, "title": "Ler sobre PATCH"}
  ],
  "meta": {"page": 2, "limit": 10, "total": 37, "totalPages": 4}
}
```

Os dados ficam em `data` e as informações da paginação em `meta`. Esse "envelope" é uma convenção, e não uma regra do REST, mas usar sempre o mesmo formato facilita a vida de quem consome a API. Defina também um `limit` máximo, como 100: sem ele, um cliente pode pedir `limit=1000000` e sobrecarregar o servidor.

A paginação por número de página é simples, mas tem um defeito: se uma tarefa nova é criada enquanto o usuário navega, os itens "escorregam" e um deles aparece em duas páginas. APIs com muito movimento, como as de redes sociais, usam a paginação por **cursor**: a resposta traz um código que aponta para o último item visto, e a próxima página começa depois dele (`?cursor=abc123`).

2.9 Versionamento

Uma API publicada tem clientes que dependem dela, e mudar o contrato pode quebrá-los. Mudanças que **não quebram** os clientes, como acrescentar um campo novo na resposta ou um *endpoint* novo, podem ser feitas a qualquer momento. Mudanças que **quebram**, como renomear ou remover um campo, mudar o tipo de um valor ou tornar obrigatório um campo que era opcional, exigem uma nova **versão** da API.

A forma mais comum e mais visível é colocar a versão na URL:

```
GET /api/v1/tasks
GET /api/v2/tasks
```

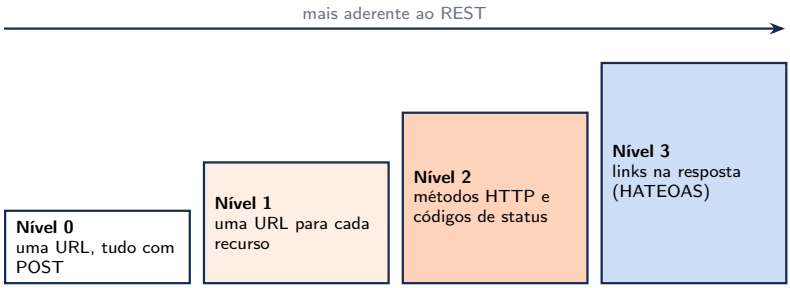
Outra opção é indicar a versão em um cabeçalho, deixando as URLs limpas. Qualquer que seja a escolha, a versão antiga deve continuar funcionando por um tempo, com aviso aos clientes, até ser desativada. A API do TaskFlow é pequena e tem um único cliente no início, então começa sem versão; se um dia precisar de uma segunda versão, ela nasce em `/api/v2`.

2.10 HATEOAS e o modelo de maturidade

A última parte da restrição de interface uniforme tem um nome comprido: **HATEOAS** (*Hypermedia as the Engine of Application State*). A ideia é que a resposta traga **links** para as ações possíveis, como uma página web traz links para as próximas páginas. O cliente descobre o que pode fazer navegando, sem precisar conhecer todas as URLs de antemão:

```
{
  "id": 42,
  "title": "Estudar REST",
  "status": "PENDING",
  "_links": {
    "self":     {"href": "/api/tasks/42"},
    "complete": {"href": "/api/tasks/42", "method": "PATCH"},
    "delete":   {"href": "/api/tasks/42", "method": "DELETE"}
  }
}
```

Leonard Richardson propôs um modelo que classifica as APIs em quatro níveis, conforme o quanto aproveitam as ideias do REST:



a maioria das APIs REST do mercado está no nível 2, como a API do TaskFlow

No **nível 0**, a API usa o HTTP só como transporte: uma URL e tudo com `POST`, como no SOAP. No **nível 1**, cada recurso tem a sua URL. No **nível 2**, os métodos e os códigos de status são usados com o seu significado. O **nível 3** acrescenta os links. A grande maioria das APIs chamadas de REST está no nível 2, que já traz quase todos os benefícios práticos, e é o nível da API do TaskFlow.

2.11 Projetando a API do TaskFlow

Antes de escrever código, uma boa prática é desenhar a API: listar os recursos, os *endpoints*, o que cada um recebe e o que devolve. Esse desenho serve de contrato entre quem faz o servidor e quem faz o cliente, que podem trabalhar ao mesmo tempo.

O TaskFlow tem dois recursos. O **usuário** tem nome, e-mail e senha. A **tarefa** tem título, descrição opcional, situação (`PENDING`, `IN_PROGRESS` ou `DONE`), prioridade (`LOW`, `MEDIUM` ou `HIGH`) e data de vencimento opcional, e pertence a um usuário. Cada pessoa só enxerga as próprias tarefas, então a URL não precisa trazer o usuário: o servidor descobre quem é pelo *token*. Por isso a API usa `/api/tasks`, e não `/api/users/7/tasks`.

Endpoint	Recebe	Sucesso
`POST /api/auth/register`	nome, e-mail e senha	`201` e o usuário, sem a senha
`POST /api/auth/login`	e-mail e senha	`200` e o *token*
`GET /api/tasks`	filtros: `status`, `priority`, `search`, `sort`, `page`, `limit`	`200`, `data` e `meta`
`POST /api/tasks`	título e, opcionalmente, os demais campos	`201`, a tarefa e `Location`
`GET /api/tasks/:id`	–	`200` e a tarefa
`PATCH /api/tasks/:id`	os campos que mudam	`200` e a tarefa atualizada
`DELETE /api/tasks/:id`	–	`204`, sem corpo

Os erros também fazem parte do contrato. Todo *endpoint* de tarefas responde `401` sem um *token* válido; os que recebem dados respondem `400` quando eles são inválidos; os que usam `:id` respondem `404` quando a tarefa não existe **ou pertence a outra pessoa**. Responder `404`, e não `403`, no segundo caso evita revelar que existe uma tarefa com aquele número. O cadastro responde `409` quando o e-mail já está em uso. Um exemplo completo de criação:

```
POST /api/tasks HTTP/1.1
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{"title": "Entregar o trabalho", "priority": "HIGH",
 "dueDate": "2026-12-01"}
```

```
HTTP/1.1 201 Created
Location: /api/tasks/cm3x9k2a0001
Content-Type: application/json

{
  "id": "cm3x9k2a0001",
  "title": "Entregar o trabalho",
  "description": null,
  "status": "PENDING",
  "priority": "HIGH",
  "dueDate": "2026-12-01T00:00:00.000Z",
  "createdAt": "2026-10-16T14:05:12.000Z"
}
```

Repare em três detalhes. O servidor preencheu sozinho os campos que o cliente não enviou: `status` começou como `PENDING`, e `description` veio como `null`. O identificador não é um número sequencial, e sim um texto aleatório (um *cuid*), gerado pelo banco no Capítulo 3; isso impede que alguém adivinhe quantas tarefas existem ou tente os números em sequência. E as datas vêm no formato ISO 8601, com o fuso UTC indicado pelo `Z`, assunto do Capítulo 5.

Erros Comuns

- **Verbos na URL.** `/getTasks` ou `/deleteTask/42` repetem o que o método já diz. Use substantivos e deixe a ação para o método.
- **Alterar dados com GET.** O GET deve ser seguro. Robôs, pré-visualizações de links e *caches* fazem GET sem pedir licença.
- **Usar POST para tudo.** A API funciona, mas perde o significado dos métodos, a idempotência e o *cache*: é o nível 0 de maturidade.
- **Mandar só parte dos campos em um PUT.** O PUT substitui o recurso inteiro. Para alterar só alguns campos, use PATCH.
- **Responder 200 para tudo, inclusive erros.** O cliente precisa ler o corpo para saber se deu certo. Use os códigos de status com o seu significado.
- **Confundir 401 e 403.** 401: "não sei quem é você". 403: "sei quem é você, mas você não pode".
- **Devolver a coleção inteira sem paginação.** Funciona com dez itens e derruba o servidor com cem mil. Defina um limite máximo.
- **Mudar o formato da resposta sem nova versão.** Renomear um campo quebra todos os clientes que dependem dele.

Capítulo 3 – Preparando o projeto

Com a API do TaskFlow desenhada no Capítulo 2, é hora de montar a base onde ela vai funcionar. Neste capítulo você vai criar o projeto com Next.js e TypeScript, criar um banco de dados PostgreSQL na nuvem, descrever as tabelas com o Prisma, aplicar a primeira migração, preencher o banco com dados de teste e escrever a primeira rota da API, que confere se tudo está ligado. Cada comando deste capítulo foi testado com as versões indicadas; se você usar versões mais novas, confira as mensagens do terminal, porque elas indicam o que mudou.

3.1 As ferramentas

Ferramenta	Papel no projeto
Node.js 20 ou mais novo	executa o JavaScript fora do navegador e traz o `npm`
Next.js 16	o *framework*: as rotas da API ficam nos *Route Handlers*, arquivos `route.ts` dentro de `app/` (o Capítulo 4 detalha)
TypeScript	JavaScript com tipos, que acusa erros antes de o código rodar
PostgreSQL	o banco de dados relacional
NeonDB	serviço que hospeda o PostgreSQL na nuvem, com plano gratuito
Prisma 7	o ORM, que traduz entre o banco e o código

Um **ORM** (*Object-Relational Mapping*, mapeamento objeto-relacional) é a camada que deixa o programa trabalhar com objetos em vez de escrever SQL à mão. Com o Prisma, buscar as tarefas pendentes de um usuário é `db.task.findMany({ where: { userId, status: "PENDING" } })`, e o resultado já vem com os tipos certos do TypeScript: o editor sabe que `title` é um texto e avisa se você escrever `titulo` por engano. O Prisma também gera o SQL que cria e altera as tabelas, a partir de um arquivo que descreve o banco.

3.2 Criando o projeto Next.js

No terminal, dentro da pasta onde você guarda os seus projetos, rode:

```
$ npx create-next-app@latest taskflow --api --ts --eslint
    --no-src-dir --import-alias "@/*" --use-npm
```

Digite tudo em uma linha só. A opção `-api` cria um projeto **só de API**, sem páginas, estilos nem React no navegador: exatamente o caso do TaskFlow. As outras opções escolhem o TypeScript, o ESLint (que aponta problemas no código), a estrutura sem a pasta `src/`, o atalho de importação `@/` e o `npm`. Com elas, o assistente não precisa fazer perguntas; se fizer alguma, aceite a sugestão. Depois, entre na pasta e inicie o servidor de desenvolvimento:

```
$ cd taskflow
$ npm run dev
```

Abra `http://localhost:3000` no navegador. Em vez de uma página, aparece um JSON: `{"message":"Hello world!"}`. É a resposta do arquivo `app/route.ts`, a primeira rota de API do projeto. O `create-next-app` também cria a pasta `app/[slug]`, um exemplo de rota com parte variável; apague essa pasta, porque o Capítulo 4 mostra como criar as rotas do TaskFlow. Para parar o servidor, aperte `Ctrl + C` no terminal. Os arquivos que importam por enquanto são estes:

Arquivo	Para que serve
`app/`	as rotas; as do TaskFlow vão ficar em `app/api/`
`package.json`	dependências e *scripts* (`dev`, `build`, `start`)
`tsconfig.json`	configuração do TypeScript, inclusive o atalho `@/`, que aponta para a raiz do projeto
`next.config.ts`	configuração do Next.js
`.gitignore`	o que não vai para o Git; já inclui `node_modules` e os arquivos `.env`

3.3 O banco de dados na nuvem com o NeonDB

O **NeonDB** oferece PostgreSQL na nuvem com um plano gratuito suficiente para estudo, e o banco fica disponível tanto no seu computador quanto, mais tarde, na Vercel. Para criar o banco:

1. Acesse `neon.tech` e crie uma conta; a forma mais rápida é entrar com o GitHub.
2. Crie um projeto chamado `taskflow`. Na região, escolha **AWS US East (N. Virginia)**, a mesma região padrão da Vercel, onde a API será publicada: banco e servidor perto um do outro deixam as consultas mais rápidas.
3. No painel do projeto, clique em **Connect**. Aparece a *connection string*, o endereço completo do banco, com usuário e senha.

O Neon mostra duas versões dessa string. Com a opção **Connection pooling** ligada, o endereço tem `-pooler` no nome do servidor; desligada, é a conexão direta. Copie as duas:

```
postgresql://usuario:senha@ep-xxx-pooler.us-east-1.aws.neon.tech/
neondb?sslmode=require

postgresql://usuario:senha@ep-xxx.us-east-1.aws.neon.tech/
neondb?sslmode=require
```

Cada string é uma linha só; aqui elas foram quebradas para caber na página. A versão com *pooler* passa por um **gerenciador de conexões**, que reaproveita poucas conexões reais para atender muitas requisições. Isso é essencial na Vercel, em que cada requisição pode rodar em uma instância diferente da API: sem o *pooler*, o banco recusaria conexões com poucos usuários. A conexão **direta** é usada pelas ferramentas do Prisma que alteram as tabelas, as migrações, que precisam de uma conexão exclusiva.

Atenção

A *connection string* contém a senha do banco. Ela nunca vai para o código nem para o GitHub: fica só no arquivo `.env`, que o `.gitignore` já protege.

Se quiser trabalhar sem internet, o Prisma também sobe um PostgreSQL local com o comando `npx prisma dev`, depois de instalado (próxima seção). Ele mostra uma *connection string* começando com `postgres://postgres:postgres@localhost`, que pode ser usada no lugar das do Neon enquanto você estuda.

3.4 Instalando e configurando o Prisma

Instale o Prisma e as bibliotecas de que ele precisa. O `@7` fixa a versão principal, para que os comandos da apostila continuem valendo:

```
$ npm install -D prisma@7 tsx
$ npm install @prisma/client@7 @prisma/adapter-pg dotenv
$ npx prisma init --datasource-provider postgresql --no-skills
```

O `prisma` é a ferramenta de linha de comando, usada só no desenvolvimento, por isso a opção `-D`; o `tsx` executa arquivos TypeScript direto, e vai rodar o *seed*. O `@prisma/client` é a biblioteca que a API usa para consultar o banco; o `@prisma/adapter-pg` é o **adaptador** que faz a conexão com o PostgreSQL, obrigatório a partir do Prisma 7; e o `dotenv` lê o arquivo `.env`. A opção `-no-skills` evita que o `init` crie pastas de instruções para assistentes de IA, que não serão usadas.

O `init` cria três arquivos: `prisma/schema.prisma`, que descreve o banco; o arquivo de configuração do Prisma, que na versão 7.10 se chama `prisma7.config.ts` (o nome `prisma.config.ts`, usado em outras versões e na documentação, também funciona); e o `.env`. Abra o `.env`, apague o exemplo e coloque as duas strings do Neon:

```
# .env -- NUNCA vai para o Git
# com -pooler: usada pela API
DATABASE_URL="postgresql://usuario:senha@ep-xxx-pooler..."
# sem -pooler: usada pelas migrações
DIRECT_URL="postgresql://usuario:senha@ep-xxx..."
```

O Next.js também lê um arquivo chamado `.env.local`, mas o Prisma não: use o `.env`, que os dois leem. Agora ajuste o arquivo de configuração para que as ferramentas do Prisma usem a conexão direta:

```
// prisma7.config.ts
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "tsx prisma/seed.ts",
  },
  datasource: {
    url: process.env["DIRECT_URL"],
  },
});
```

O `import "dotenv/config"` carrega o `.env`; sem ele, as variáveis ficam vazias, porque o Prisma 7 não lê o arquivo sozinho. A linha `seed` indica o comando que preenche o banco com dados de teste (Seção 3.8). Se estiver usando o banco local do `prisma dev`, coloque a mesma string nas duas variáveis.

3.5 O esquema do banco

O `schema.prisma` descreve as tabelas em uma linguagem própria, simples de ler. Substitua o conteúdo dele por este:

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client"
  output   = "../app/generated/prisma"
}

datasource db {
  provider = "postgresql"
}

model User {
  id        String   @id @default(cuid())
  name      String
  email     String   @unique
  password  String   // o hash da senha, nunca a senha
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  tasks     Task[]

  @@map("users")
}

model Task {
  id          String     @id @default(cuid())
  title       String
  description String?
  status      TaskStatus @default(PENDING)
  priority    Priority   @default(MEDIUM)
  dueDate     DateTime?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  userId      String
  user        User       @relation(fields: [userId], references: [id],
  onDelete: Cascade)

  @@index([userId])
  @@map("tasks")
}

enum TaskStatus {
  PENDING
  IN_PROGRESS
```

```
  DONE
}

enum Priority {
  LOW
  MEDIUM
  HIGH
}
```

A linha do campo `user`, que começa com `user User @relation`, é uma só no arquivo; aqui ela foi quebrada para caber na página. Digite-a em uma linha: o Prisma não aceita um atributo dividido em duas linhas e acusa o erro `P1012`.

O bloco `generator` diz onde o Prisma gera o código do cliente: na pasta `app/generated/prisma`, que não vai para o Git (o `init` já a acrescentou ao `.gitignore`). Cada `model` vira uma tabela, e cada linha dentro dele, uma coluna. Os símbolos que aparecem são estes:

Símbolo	Significado
`@id`	chave primária
`@default(cuid())`	o banco gera um identificador aleatório, como `cm3x9k2a0001`
`@unique`	não pode haver dois iguais; dois usuários não têm o mesmo e-mail
`String?`	o `?` torna o campo opcional (aceita `null`)
`@default(now())`	preenchido com a data e a hora da criação
`@updatedAt`	atualizado sozinho a cada alteração
`Task[]`	um usuário tem várias tarefas (lado "um" da relação)
`@relation(...)`	a chave estrangeira: `userId` aponta para o `id` do usuário
`onDelete: Cascade`	ao excluir um usuário, as tarefas dele são excluídas também
`@@index([userId])`	índice para buscar rápido as tarefas de um usuário
`@@map("tasks")`	o nome da tabela no banco
`enum`	lista fechada de valores permitidos

O índice em `userId` tem motivo: quase toda consulta da API filtra as tarefas pelo dono, e sem o índice o banco teria de percorrer a tabela inteira a cada requisição.

3.6 A primeira migração

Uma **migração** é um arquivo SQL que leva o banco de um estado a outro, como "criar a tabela de tarefas". As migrações ficam guardadas no projeto e vão para o Git, assim qualquer pessoa da equipe, e o servidor de produção, chega exatamente ao mesmo banco. Crie e aplique a primeira:

```
$ npx prisma migrate dev --name init
$ npx prisma generate
```

O `migrate dev` compara o esquema com o banco, escreve o SQL em `prisma/migrations/<data>_i` e o executa. Um trecho do arquivo gerado:

```
CREATE TYPE "TaskStatus"
  AS ENUM ('PENDING', 'IN_PROGRESS', 'DONE');

CREATE TABLE "tasks" (
    "id" TEXT NOT NULL,
    "title" TEXT NOT NULL,
    "description" TEXT,
    "status" "TaskStatus" NOT NULL DEFAULT 'PENDING',
    ...
    "userId" TEXT NOT NULL,
    CONSTRAINT "tasks_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
CREATE INDEX "tasks_userId_idx" ON "tasks"("userId");
ALTER TABLE "tasks" ADD CONSTRAINT "tasks_userId_fkey"
  FOREIGN KEY ("userId") REFERENCES "users"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

Vale ler o SQL: ele mostra que o `@unique` virou um índice único e que o `onDelete: Cascade` virou `ON DELETE CASCADE`. O `generate` cria o código do cliente, com os tipos do TypeScript, em `app/generated/prisma`. No Prisma 7, ele não roda mais sozinho depois da migração: **sempre que alterar o esquema, rode os dois comandos**. Mais tarde, quando o esquema mudar, você cria outra migração com outro nome, como `-name add-categorias`, e o Prisma escreve só a diferença.

3.7 Uma conexão para toda a API

A API vai consultar o banco em vários arquivos, e todos devem usar o mesmo cliente do Prisma. Crie a pasta `lib` na raiz do projeto e, dentro dela, o arquivo `db.ts`:

```
// lib/db.ts -- uma única conexão com o banco para toda a API
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "@/app/generated/prisma/client";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
});

// Em desenvolvimento, o Next.js recarrega os arquivos a cada
// alteração; guardar o cliente em globalThis evita abrir uma
// conexão nova a cada recarga.
const globalParaPrisma = globalThis as unknown as {
  prisma?: PrismaClient;
};

export const db =
  globalParaPrisma.prisma ?? new PrismaClient({ adapter });
```

```
if (process.env.NODE_ENV !== "production") {
  globalParaPrisma.prisma = db;
}
```

O cliente é importado da pasta gerada, `@/app/generated/prisma/client`, e não do pacote `@prisma/client`, como em versões antigas do Prisma. O adaptador recebe a `DATABASE_URL`, a do *pooler*. A parte do `globalThis` resolve um problema do desenvolvimento: a cada arquivo salvo, o Next.js recarrega os módulos, e cada recarga criaria um cliente novo, com conexões novas, até esgotar o limite do banco. Guardando o cliente em uma variável global, a recarga reaproveita o mesmo. Em produção não há recarga, então o `if` não guarda nada. Esse padrão, de ter um único objeto compartilhado, se chama ***singleton***.

O operador `??` devolve o valor da esquerda quando ele existe, e o da direita quando é `null` ou `undefined`: "use o cliente guardado ou, se não houver, crie um".

3.8 Dados de teste e o Prisma Studio

Para testar a API, é bom ter dados no banco desde o início. O ***seed*** é um *script* que os cria. Para criar o usuário de teste com a senha protegida, instale o `bcryptjs`, que transforma a senha em um *hash*, assunto do Capítulo 9:

```
$ npm install bcryptjs
```

```
// prisma/seed.ts -- dados de teste para o desenvolvimento
import "dotenv/config";
import bcrypt from "bcryptjs";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../app/generated/prisma/client";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
});
const db = new PrismaClient({ adapter });

async function main() {
  const senha = await bcrypt.hash("senha123", 10);

  const usuario = await db.user.upsert({
    where: { email: "demo@taskflow.com" },
    update: {},
    create: {
      name: "Usuária Demo",
      email: "demo@taskflow.com",
      password: senha,
      tasks: {
        create: [
          { title: "Ler o Capítulo 1", status: "DONE" },
          { title: "Projetar a API", status: "DONE" },
```

```
          { title: "Configurar o Prisma", status: "IN_PROGRESS",
            priority: "HIGH" },
          { title: "Criar as rotas", priority: "HIGH" },
          { title: "Publicar na Vercel", priority: "LOW" },
        ],
      },
    },
    include: { tasks: true },
  });

  const total = usuario.tasks.length;
  console.log(`${usuario.email}: ${total} tarefas`);
}

main()
  .catch((erro) => {
    console.error(erro);
    process.exit(1);
  })
  .finally(() => db.$disconnect());
```

O *seed* roda fora do Next.js, por isso cria o próprio cliente e importa a pasta gerada pelo caminho relativo (`../app/...`), e não pelo atalho `@/`. O `upsert` é "atualize se existir, crie se não existir": procura o usuário pelo e-mail e, como `update` está vazio, não muda nada se ele já existir. Assim, o *seed* pode ser rodado várias vezes sem duplicar dados. As tarefas são criadas junto com o usuário, dentro de `tasks: { create: [...] }`, e os campos não informados recebem os valores padrão do esquema. Rode:

```
$ npx prisma db seed
Running seed command `tsx prisma/seed.ts` ...
demo@taskflow.com: 5 tarefas
```

Para ver o banco, abra o **Prisma Studio** com `npx prisma studio`. Ele abre no navegador uma tela parecida com uma planilha, em que você consulta e edita as tabelas. O painel do Neon também tem um editor de tabelas e um editor de SQL.

3.9 A primeira rota: conferindo a conexão

Para terminar, uma rota que confirma que a API fala com o banco. No App Router, uma rota de API é um arquivo `route.ts`, e o caminho das pastas vira a URL, como as páginas. Crie `app/api/health/route.ts`:

```
// app/api/health/route.ts -- confere se a API fala com o banco
import { db } from "@/lib/db";

export async function GET() {
  const tarefas = await db.task.count();
  return Response.json({ status: "ok", tarefas });
}
```

A função se chama `GET` porque responde ao método GET; o Capítulo 4 mostra as outras. `Response.json` monta uma resposta com o corpo em JSON e o cabeçalho `Content-Type` certo. Com o `npm run dev` rodando, teste no navegador ou no terminal:

```
$ curl -i http://localhost:3000/api/health
HTTP/1.1 200 OK
content-type: application/json

{"status":"ok","tarefas":5}
```

Se aparecer `"tarefas":5`, o caminho inteiro funciona: a requisição chegou ao Next.js, a rota usou o cliente do Prisma, o adaptador se conectou ao Neon e a contagem voltou em JSON. Rotas como esta, chamadas de *health check*, são usadas de verdade em produção, para que os serviços de monitoramento saibam se a API está no ar.

3.10 A estrutura do projeto

```
taskflow/
  app/
    api/
      health/route.ts     GET /api/health
    generated/prisma/     cliente gerado (fora do Git)
    route.ts              GET / (a mensagem de boas-vindas)
  lib/
    db.ts                 o cliente do Prisma (singleton)
  prisma/
    migrations/           o histórico do banco (vai para o Git)
    schema.prisma         as tabelas
    seed.ts               dados de teste
  .env                    senhas e strings (fora do Git)
  prisma7.config.ts       configuração do Prisma
  package.json
```

A partir do próximo capítulo, as rotas do TaskFlow vão sendo criadas dentro de `app/api/`, e as funções de apoio, como a validação e a autenticação, dentro de `lib/`.

Erros Comuns

- **Esquecer o `prisma generate`.** Depois de alterar o esquema e migrar, o código antigo do cliente continua lá, e o TypeScript não reconhece os campos novos. No Prisma 7, a migração não gera o cliente sozinha.
- **Importar o cliente de `@prisma/client`.** Com o gerador `prisma-client`, o cliente fica na pasta indicada em `output`: importe de `@/app/generated/prisma/client`.
- **Esquecer o `import "dotenv/config"`.** O Prisma 7 não lê o `.env` sozinho, e o erro diz que a URL do banco está vazia.
- **Colocar as strings no `.env.local`.** O Next.js lê esse arquivo, mas as ferramentas do Prisma, não. Use o `.env`.
- **Usar a string do *pooler* nas migrações.** As migrações precisam da conexão direta, sem `-pooler`, na `DIRECT_URL`.

- **Criar um `new PrismaClient()` em cada arquivo.** Cada cliente abre conexões próprias. Importe sempre o `db` de `lib/db.ts`.
- **Enviar o `.env` para o GitHub.** Se acontecer, troque a senha do banco no Neon imediatamente: apagar o arquivo depois não basta, porque ele fica no histórico do Git.
- **Editar um arquivo de migração já aplicado.** O Prisma confere as migrações aplicadas; para mudar o banco, altere o esquema e crie uma migração nova.

Capítulo 4 – Roteamento e endpoints

No Capítulo 3, a rota `/api/health` mostrou que basta um arquivo para criar um *endpoint*. Neste capítulo você vai entender a regra que liga cada URL a uma função do código, ler tudo o que chega em uma requisição (parâmetros, cabeçalhos e corpo), montar respostas com o status certo e criar as primeiras rotas de verdade do TaskFlow: listar, criar, buscar e excluir tarefas. No final, vai conhecer o que o Next.js faz sozinho, como as respostas a `OPTIONS` e o status `405`, e as formas práticas de testar uma API durante o desenvolvimento.

4.1 Do endereço ao código

Roteamento é o mapeamento entre uma requisição (método e URL) e a função que vai atendê-la. Em muitos *frameworks*, como o Express, as rotas são escritas em código, uma por linha: `app.get("/api/tasks", listar)`. No Next.js, o mapeamento é feito pelas **pastas**: cada pasta dentro de `app/` é um pedaço da URL, e um arquivo `route.ts` transforma aquela pasta em *endpoint*.

Arquivo	URL
`app/route.ts`	`/`
`app/api/health/route.ts`	`/api/health`
`app/api/tasks/route.ts`	`/api/tasks`
`app/api/tasks/[id]/route.ts`	`/api/tasks/42`, `/api/tasks/abc`...
`app/api/auth/login/route.ts`	`/api/auth/login`

Dentro do `route.ts`, cada função exportada com o nome de um método HTTP atende àquele método: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD` e `OPTIONS`. O nome precisa estar em maiúsculas. Assim, o mapeamento completo de um *endpoint* é "pasta + nome da função": `POST /api/tasks` é a função `POST` do arquivo `app/api/tasks/route.ts`. Esses arquivos se chamam ***Route Handlers***.

Uma pasta pode ter um `route.ts` ou uma página (`page.tsx`), mas não os dois: o Next.js não saberia se deve devolver HTML ou executar a rota.

4.2 Lendo a requisição

Toda função de rota recebe a requisição como primeiro parâmetro. O tipo `NextRequest` é o `Request` padrão da web, o mesmo do `fetch`, com alguns atalhos a mais. Ela traz tudo o que o cliente enviou:

O quê	Como ler
query string	`request.nextUrl.searchParams.get("status")`
cabeçalho	`request.headers.get("authorization")`
corpo em JSON	`await request.json()`
corpo em texto	`await request.text()`
corpo de formulário	`await request.formData()`
parte variável da URL	`(await contexto.params).id` (Seção 4.6)
método e URL	`request.method` e `request.url`

Três detalhes evitam surpresas. O `searchParams.get` devolve sempre um texto, ou `null` se o parâmetro não veio: `?page=2` chega como `"2"`, e não como o número 2. O nome dos cabeçalhos não diferencia maiúsculas de minúsculas, então `get("Authorization")`

e `get("authorization")` dão no mesmo. E o corpo só pode ser lido **uma vez**: depois de um `request.json()`, um `request.text()` falha.

4.3 Montando a resposta

A função devolve um `Response`, também o padrão da web. As três formas que aparecem em quase toda API são estas:

```
// 200 com corpo JSON (o status padrão é 200)
return Response.json({ data: tarefas });

// outro status e cabeçalhos extras
return Response.json(tarefa, {
  status: 201,
  headers: { Location: `/api/tasks/${tarefa.id}` },
});

// sem corpo: o 204 não pode ter corpo
return new Response(null, { status: 204 });
```

O `Response.json` transforma o objeto em texto JSON e acrescenta o cabeçalho `Content-Type: application/json`. Você vai encontrar também o `NextResponse.json`, de `next/server`, que faz o mesmo e tem recursos extras para *cookies* e redirecionamentos; para uma API, o `Response` padrão basta.

4.4 Um usuário provisório

Cada pessoa só pode ver as próprias tarefas, então toda rota precisa saber quem está fazendo a requisição. Isso depende da autenticação, que chega no Capítulo 9. Até lá, uma função provisória finge que toda requisição vem da usuária de teste criada pelo *seed*:

```
// lib/usuario-atual.ts -- PROVISÓRIO: até o Capítulo 9, toda
// requisição age como a usuária de teste criada pelo seed.
import { db } from "@/lib/db";

export async function usuarioAtual() {
  return db.user.findUniqueOrThrow({
    where: { email: "demo@taskflow.com" },
    select: { id: true, name: true },
  });
}
```

O `findUniqueOrThrow` busca pelo e-mail, que é único, e lança um erro se não encontrar. O `select` traz só o identificador e o nome: a senha nunca precisa sair do banco aqui. As rotas vão chamar `usuarioAtual()` sem saber como ela funciona por dentro. Quando a autenticação estiver pronta, só esta função muda, e nenhuma rota precisa ser alterada. Isolar o que vai mudar atrás de uma função é uma das técnicas mais úteis de organização de código.

4.5 A coleção de tarefas

O arquivo `app/api/tasks/route.ts` atende à coleção: listar (`GET`) e criar (`POST`).

```
// app/api/tasks/route.ts -- a coleção de tarefas
import type { NextRequest } from "next/server";
import { db } from "@/lib/db";
import { TaskStatus } from "@/app/generated/prisma/client";
import { usuarioAtual } from "@/lib/usuario-atual";

// GET /api/tasks e GET /api/tasks?status=PENDING
export async function GET(request: NextRequest) {
  const usuario = await usuarioAtual();
  const status = request.nextUrl.searchParams.get("status");

  if (status && !Object.hasOwn(TaskStatus, status)) {
    return Response.json(
      { erro: "status deve ser PENDING, IN_PROGRESS ou DONE" },
      { status: 400 },
    );
  }

  const tarefas = await db.task.findMany({
    where: {
      userId: usuario.id,
      ...(status && { status: status as TaskStatus }),
    },
    orderBy: { createdAt: "desc" },
  });

  return Response.json({ data: tarefas });
}
```

O Prisma gera, para cada `enum` do esquema, um objeto com os valores permitidos: `TaskStatus` é `{ PENDING: "PENDING", IN_PROGRESS: ..., DONE: ... }`. O `Object.hasOwn(TaskStatus, status)` confere se o texto recebido é uma dessas chaves. Repare que não foi usado `status in TaskStatus`: o operador `in` também enxerga as propriedades herdadas de todo objeto, e `?status=toString` passaria pela verificação e quebraria a consulta.

O filtro usa uma construção comum em JavaScript: `...(status && { status })`. Se `status` for `null`, a expressão vale `null`, e espalhar `null` em um objeto não acrescenta nada; se houver um valor, o campo `status` entra no `where`. O `as TaskStatus` diz ao TypeScript que o texto já foi conferido. Por fim, `orderBy` lista as tarefas mais recentes primeiro, e o resultado vai dentro de `data`, o envelope combinado no Capítulo 2. A criação fica no mesmo arquivo:

```
// POST /api/tasks  -- corpo: {"title": "..."}
export async function POST(request: NextRequest) {
  const usuario = await usuarioAtual();
  const corpo = await request.json().catch(() => null);
```

```
  if (typeof corpo?.title !== "string" || !corpo.title.trim()) {
    return Response.json(
      { erro: "O título é obrigatório." },
      { status: 400 },
    );
  }

  const tarefa = await db.task.create({
    data: { title: corpo.title.trim(), userId: usuario.id },
  });

  return Response.json(tarefa, {
    status: 201,
    headers: { Location: `/api/tasks/${tarefa.id}` },
  });
}
```

Se o corpo não for um JSON válido, `request.json()` lança um erro, e a API responderia `500`, como se a falha fosse do servidor. O `.catch(() => null)` transforma esse caso em `null`, que cai na verificação seguinte e recebe um `400`, o código certo para um erro do cliente. O `corpo?.title` usa o **encadeamento opcional**: se `corpo` for `null`, o resultado é `undefined` em vez de um erro. A verificação aceita só um texto que não fique vazio depois do `trim`, que tira os espaços das pontas.

Esta verificação "à mão" cobre só o título. Conferir todos os campos (prioridade, data, tamanho do texto) desse jeito seria longo e fácil de errar; o Capítulo 6 substitui tudo por um esquema do Zod, e o Capítulo 7 completa a criação com os demais campos.

4.6 Segmentos dinâmicos: uma tarefa

Para atender `/api/tasks/42`, `/api/tasks/abc` e qualquer outro identificador, o nome da pasta vai entre colchetes: `app/api/tasks/[id]/`. É um **segmento dinâmico**, e o valor que veio na URL chega à função no segundo parâmetro, o **contexto**:

```
// app/api/tasks/[id]/route.ts -- uma tarefa
import type { NextRequest } from "next/server";
import { db } from "@/lib/db";
import { usuarioAtual } from "@/lib/usuario-atual";

// GET /api/tasks/:id
export async function GET(
  request: NextRequest,
  contexto: RouteContext<"/api/tasks/[id]">,
) {
  const { id } = await contexto.params;
  const usuario = await usuarioAtual();

  const tarefa = await db.task.findFirst({
    where: { id, userId: usuario.id },
  });
```

```
  if (!tarefa) {
    return Response.json(
      { erro: "Tarefa não encontrada." },
      { status: 404 },
    );
  }
  return Response.json(tarefa);
}
```

O `RouteContext<"/api/tasks/[id]">` é um tipo que o próprio Next.js gera a partir das pastas do projeto, ao rodar `npm run dev` ou `npm run build`: ele sabe que essa rota tem um parâmetro `id`. Os parâmetros chegam como uma *Promise*, por isso o `await contexto.params`; até a versão 14 do Next.js, eles eram um objeto comum, e você vai encontrar exemplos antigos sem o `await`.

A consulta é a parte mais importante. Buscar só pelo `id`, com `findUnique({ where: { id } })`, devolveria a tarefa de qualquer pessoa que soubesse o identificador. O `findFirst` com `id e userId` só encontra a tarefa se ela pertencer a quem pediu. Se for de outra pessoa, o resultado é o mesmo de uma tarefa que não existe: `404`. Como combinado no Capítulo 2, isso não revela a existência de tarefas alheias. A exclusão segue a mesma regra:

```
// DELETE /api/tasks/:id
export async function DELETE(
  request: NextRequest,
  contexto: RouteContext<"/api/tasks/[id]">,
) {
  const { id } = await contexto.params;
  const usuario = await usuarioAtual();

  const { count } = await db.task.deleteMany({
    where: { id, userId: usuario.id },
  });

  if (count === 0) {
    return Response.json(
      { erro: "Tarefa não encontrada." },
      { status: 404 },
    );
  }
  return new Response(null, { status: 204 });
}
```

O `deleteMany` aceita qualquer filtro e devolve quantos registros apagou. Com o filtro por `id` e dono, o resultado é 1 (apagou) ou 0 (não havia tarefa daquele dono com aquele `id`). Fazer tudo em uma única consulta evita buscar primeiro e apagar depois. O parâmetro `request` não é usado em nenhuma das duas funções, mas precisa estar lá, porque o contexto é sempre o segundo parâmetro. A edição (`PATCH`) entra no Capítulo 7, junto com a validação de todos os campos.

4.7 O que o Next.js faz sozinho

Com os dois arquivos prontos, alguns comportamentos aparecem sem nenhuma linha de código a mais. Estes resultados foram obtidos com o servidor rodando:

Requisição	Resposta
`HEAD /api/tasks`	`200`, só com os cabeçalhos: o Next.js usa a função `GET`
`OPTIONS /api/tasks`	`204` com o cabeçalho `allow: GET, HEAD, OPTIONS, POST`
`PATCH /api/tasks/42`	`405 Method Not Allowed`: o arquivo não exporta `PATCH`
`GET /api/tarefas`	`404`: não há pasta com esse nome

Ou seja: o `405` aparece quando a URL existe mas o método não, e o `404`, quando nem a URL existe, exatamente como manda o HTTP.

4.8 Outras convenções de pastas

Além do segmento dinâmico, o App Router tem outras convenções de nome de pasta que aparecem em APIs maiores:

Pasta	Efeito
`[id]`	um segmento variável: `/tasks/42`
`[...caminho]`	um ou mais segmentos: `/arquivos/a/b/c`; o parâmetro vira a lista `["a", "b", "c"]`
`[[...caminho]]`	como o anterior, mas também atende à URL sem nenhum segmento: `/arquivos`
`(grupo)`	organiza as pastas sem entrar na URL: `app/(admin)/api/users` atende `/api/users`
`_privada`	ignorada pelo roteamento; serve para guardar arquivos auxiliares ao lado das rotas

Quando uma URL combina com mais de uma pasta, vence a mais específica: com `app/api/tasks/[id]` e `app/api/tasks/resumo`, a URL `/api/tasks/resumo` vai para a pasta `resumo`, e não para o `[id]`.

4.9 Testando as rotas

O navegador só faz `GET` pela barra de endereços. Para os outros métodos, há três caminhos comuns. O primeiro é o `curl`, no terminal:

```
$ curl http://localhost:3000/api/tasks?status=DONE
$ curl -i -X POST http://localhost:3000/api/tasks \
    -H "Content-Type: application/json" \
    -d '{"title": "Testar as rotas"}'
$ curl -i -X DELETE http://localhost:3000/api/tasks/cm3x9k2a0001
```

A opção `-X` escolhe o método, `-H` acrescenta um cabeçalho e `-d` envia o corpo. A barra invertida no fim da linha continua o comando na linha seguinte; ela funciona no

terminal Git Bash, no macOS e no Linux. No PowerShell, as aspas dentro do JSON exigem cuidados extras, por isso, no Windows, prefira o segundo caminho.

O segundo é a extensão **REST Client**, do VS Code. Crie um arquivo com a extensão `.http`, escreva as requisições separadas por `###` e clique em *Send Request* acima de cada uma:

```
### Listar as tarefas pendentes
GET http://localhost:3000/api/tasks?status=PENDING

### Criar uma tarefa
POST http://localhost:3000/api/tasks
Content-Type: application/json

{"title": "Testar as rotas"}

### Criar sem título (espera 400)
POST http://localhost:3000/api/tasks
Content-Type: application/json

{}
```

O arquivo fica no projeto e documenta os testes: qualquer pessoa da equipe consegue repeti-los. O terceiro caminho são programas com interface gráfica, como o **Postman**, assunto do Capítulo 10. Seja qual for a ferramenta, teste sempre os casos de erro, e não só o caminho feliz: título vazio, JSON quebrado, `status` inválido, identificador inexistente e exclusão repetida.

4.10 Como está o projeto

```
app/
  api/
    health/route.ts     GET
    tasks/
      route.ts          GET (com ?status), POST
      [id]/route.ts     GET, DELETE
lib/
  db.ts
  usuario-atual.ts      provisório, até o Capítulo 9
testes.http             requisições para o REST Client
```

Repare que as respostas ainda trazem todos os campos da tarefa, inclusive o `userId`, que não interessa ao cliente, e que cada rota escreve a sua própria mensagem de erro. O Capítulo 5 cuida do formato dos dados que entram e saem, e o Capítulo 8 padroniza os erros.

Erros Comuns

- **Nome da função em minúsculas.** `export async function get()` não é reconhecido, e a rota responde `405`. O nome é o do método, em maiúsculas.

- **`route.ts` e `page.tsx` na mesma pasta.** O Next.js acusa conflito. Rotas de API ficam em pastas próprias, como `app/api/`.
- **Esquecer o `await` em `contexto.params`.** Os parâmetros são uma *Promise*; sem o `await`, `id` fica `undefined`.
- **Buscar só pelo `id`.** Sem o `userId` no filtro, qualquer pessoa lê ou apaga a tarefa de outra. Essa falha é tão comum que lidera a lista de riscos de APIs da OWASP, estudada no Capítulo 12.
- **Deixar o JSON inválido virar 500.** Trate o erro de `request.json()` e responda `400`.
- **Tratar a *query string* como número.** `searchParams.get("page")` devolve o texto `"2"`; converta com `Number()` e confira o resultado.
- **Mandar corpo com o status 204.** O 204 significa "sem conteúdo"; use `new Response(null, { status: 204 })`.
- **Esquecer o `return`.** Se a função monta a resposta de erro mas não a devolve, o código continua e executa a consulta mesmo assim.

Capítulo 5 – JSON, XML e serialização

Uma API recebe e devolve dados pela rede, e a rede só transporta texto (ou, mais precisamente, bytes). Os objetos que existem na memória do programa, com as suas datas, listas e números, precisam ser transformados em texto para viajar e reconstruídos do outro lado. Neste capítulo você vai entender esse processo, chamado serialização, conhecer em detalhe os dois formatos mais usados, o JSON e o XML, e ver as armadilhas que mais causam erros: datas, valores que o JSON não representa e campos que não deveriam sair da API. No final, o TaskFlow passa a controlar exatamente os campos que devolve e a responder também em XML a quem pedir.

5.1 Serializar e desserializar

Serializar é transformar um objeto da memória em uma sequência de texto ou bytes que possa ser guardada ou transmitida. **Desserializar** é o caminho inverso: ler o texto e reconstruir o objeto. Em JavaScript, as duas operações com JSON são `JSON.stringify` e `JSON.parse`:

```
const tarefa = { title: "Estudar JSON", done: false };

const texto = JSON.stringify(tarefa);   // serializar
console.log(texto);
// {"title":"Estudar JSON","done":false}
console.log(typeof texto);  // string

const deVolta = JSON.parse(texto);      // desserializar
console.log(deVolta.title); // Estudar JSON
```

Em uma API, isso acontece em toda requisição. Quando a rota faz `await request.json()`, o Next.js lê o corpo como texto e aplica `JSON.parse`; quando ela devolve `Response.json(tarefa)`, ele aplica `JSON.stringify` e põe o resultado no corpo. Do outro lado, o `fetch` do cliente faz o mesmo com `resposta.json()`.

O `JSON.stringify` aceita dois parâmetros extras. O terceiro indica a indentação, útil para ler ou gravar arquivos: `JSON.stringify(obj, null, 2)` põe cada campo em uma linha, com dois espaços. O segundo, que aqui ficou `null`, permite filtrar ou transformar campos.

5.2 JSON em detalhe

O **JSON** (*JavaScript Object Notation*) nasceu da sintaxe de objetos do JavaScript, mas é um formato independente, padronizado na RFC 8259 e usado por praticamente todas as linguagens. Ele tem só seis tipos de valor:

Tipo	Exemplo	Observação
texto	`"Estudar"`	sempre entre aspas duplas
número	`42`, `3.14`, `-1e3`	sem distinção entre inteiro e decimal
booleano	`true`, `false`	em minúsculas
nulo	`null`	ausência de valor
objeto	`{"a": 1}`	pares nome-valor; o nome sempre entre aspas duplas
lista	`[1, "dois", null]`	valores em ordem, de qualquer tipo

O JSON é mais rígido que o JavaScript. Não aceita aspas simples, vírgula depois do último item, comentários, nem nomes de campo sem aspas. Um JSON escrito à mão com um desses detalhes faz o `JSON.parse` falhar:

```
JSON.parse("{'title': 'x'}");
// SyntaxError: Expected property name or '}' in JSON
JSON.parse('{"a": 1,}');
// SyntaxError: Expected double-quoted property name in JSON
```

Também há valores do JavaScript que o JSON não sabe representar. O `JSON.stringify` não avisa: ele simplesmente descarta ou troca esses valores, e o dado chega diferente do outro lado.

```
const tarefa = {
  title: "Estudar JSON",
  dueDate: new Date("2026-12-01T15:30:00Z"),
  description: undefined,
  nota: NaN,
  salvar() {},
};
console.log(JSON.stringify(tarefa));
// {"title":"Estudar JSON","dueDate":"2026-12-01T15:30:00.000Z",
//  "nota":null}
```

Valor em JavaScript	O que vira no JSON
`Date`	texto no formato ISO 8601 (e não volta a ser data no `parse`)
`undefined` e funções	o campo desaparece
`NaN` e `Infinity`	`null`
`Map` e `Set`	`{}`, vazios
`BigInt` (`10n`)	erro: `TypeError: Do not know how to serialize a BigInt`

Por isso, em uma API, use `null` para dizer "sem valor" (e não `undefined`, que some), e converta `Map`, `Set` e `BigInt` para objetos, listas e textos antes de responder. Há ainda o limite dos números: o JSON não restringe o tamanho, mas o JavaScript só representa inteiros com exatidão até cerca de 9 quatrilhões (`Number.MAX_SAFE_INTEGER`). Identificadores muito grandes, como os de algumas redes sociais, são enviados como texto por esse motivo.

5.3 Datas: a armadilha clássica

O JSON não tem um tipo de data. A convenção adotada por quase todas as APIs é enviar a data como texto no formato **ISO 8601**, o mesmo que o `Date` produz ao ser serializado:

```
"2026-12-01T15:30:00.000Z"
 ano-mês-dia T hora:min:seg.ms  Z = horário UTC
```

A ordem ano-mês-dia elimina a confusão entre o formato brasileiro (dia/mês) e o americano (mês/dia), e ainda faz as datas ficarem em ordem quando ordenadas como texto. O `Z` no final indica o **UTC**, o horário de referência mundial, de onde se calculam os fusos: o horário de Brasília é UTC menos 3 horas, escrito `-03:00`. A boa prática é **guardar e transmitir em UTC** e converter para o fuso do usuário só na hora de mostrar. Os problemas aparecem quando a data é lida sem pensar no fuso:

```
// rodando em um computador no Brasil
const d = new Date("2026-12-01");
d.toISOString();          // 2026-12-01T00:00:00.000Z
d.toLocaleString("pt-BR");    // 30/11/2026, 21:00:00

new Date("2026-12-01T00:00:00").toISOString();
// 2026-12-01T03:00:00.000Z

new Date("01/12/2026").toISOString();
// 2026-01-12T03:00:00.000Z   (12 de janeiro!)
new Date("31/12/2026").getTime();   // NaN
```

Os resultados acima foram obtidos com o fuso de São Paulo. Uma data sem horário, como `"2026-12-01"`, é lida como meia-noite em UTC; mostrada no Brasil, vira 21h do dia **anterior**, e a tarefa que vence em 1º de dezembro aparece como vencendo em 30 de novembro. Já a mesma data com horário e sem o `Z` é lida no fuso do computador. E o formato com barras é interpretado à americana: `01/12` vira 12 de janeiro, e `31/12` nem é uma data válida.

Para o campo `dueDate` do TaskFlow, que é uma data de vencimento, as regras ficam assim: o cliente envia `"2026-12-01"`, a API guarda meia-noite em UTC e devolve `"2026-12-01T00:00:00.000Z"`, e quem exibe a data mostra apenas a parte do dia, lida em UTC, como em `d.toLocaleDateString("pt-BR", { timeZone: "UTC" })`, que dá `01/12/2026`. Nunca aceite datas com barras em uma API: exija o formato ISO, e o Capítulo 6 mostra como validar isso.

5.4 Nomes dos campos

Duas convenções disputam o nome dos campos em JSON: **camelCase** (`dueDate`, `createdAt`), a do JavaScript, e **snake_case** (`due_date`, `created_at`), comum em APIs feitas em Python e Ruby. Nenhuma é melhor; o que importa é escolher uma e usá-la em toda a API. O TaskFlow usa camelCase, que é o padrão do Prisma e do TypeScript, e por isso os nomes do banco chegam à resposta sem nenhuma conversão.

Outra decisão de projeto: os campos da API do TaskFlow estão em inglês (`title`, `status`), enquanto as mensagens de erro, lidas por pessoas, estão em português. É

o padrão mais comum no mercado, porque bibliotecas, documentações e colegas de outros países esperam nomes em inglês. Também aqui, consistência vale mais que a escolha em si.

5.5 Controlando o que sai da API

No Capítulo 4, as rotas devolviam a tarefa exatamente como veio do banco, inclusive o `userId`. Devolver o registro inteiro é um hábito perigoso: no dia em que alguém acrescentar ao banco um campo interno, ele vaza para a API sem que ninguém perceba. No caso do usuário, o campo seria a senha. A forma segura é definir **explicitamente** quais campos formam a representação pública do recurso:

```
// lib/tarefas.ts -- o formato público de uma tarefa
import type { Prisma } from "@/app/generated/prisma/client";

// Os campos que a API devolve. O userId fica de fora: o cliente
// já sabe que as tarefas são dele.
export const camposTarefa = {
  id: true,
  title: true,
  description: true,
  status: true,
  priority: true,
  dueDate: true,
  createdAt: true,
  updatedAt: true,
} satisfies Prisma.TaskSelect;
```

O objeto segue o formato do `select` do Prisma, e o `satisfies Prisma.TaskSelect` faz o TypeScript conferir que cada nome existe no modelo: um erro de digitação, como `titel`, é acusado no editor. Agora, todas as consultas que devolvem tarefas usam o mesmo `select`:

```
// em app/api/tasks/route.ts
import { camposTarefa } from "@/lib/tarefas";

const tarefas = await db.task.findMany({
  where: { userId: usuario.id },
  select: camposTarefa,
  orderBy: { createdAt: "desc" },
});
```

Faça o mesmo no `create` do `POST` e no `findFirst` do `GET` de uma tarefa. Além da segurança, o `select` tem duas vantagens: o banco envia menos dados, e o tipo do resultado, calculado pelo Prisma, passa a ter só esses campos. O conjunto de campos que uma API aceita ou devolve é chamado de **DTO** (*Data Transfer Object*, objeto de transferência de dados), e separá-lo do formato do banco é uma prática de qualquer API profissional.

5.6 XML

O **XML** (*eXtensible Markup Language*) organiza os dados em **elementos** delimitados por marcas de abertura e fechamento, como o HTML, só que com nomes livres. Foi o formato dominante na troca de dados entre sistemas nos anos 2000 e continua presente no SOAP, na Nota Fiscal Eletrônica, em arquivos de configuração e em formatos de documento, como o `.docx`. O ViaCEP, visto no Capítulo 1, devolve XML se a URL terminar em `/xml/`:

```
<?xml version="1.0" encoding="UTF-8"?>
<xmlcep>
  <cep>01001-000</cep>
  <logradouro>Praça da Sé</logradouro>
  <complemento>lado ímpar</complemento>
  <bairro>Sé</bairro>
  <localidade>São Paulo</localidade>
  <uf>SP</uf>
</xmlcep>
```

A primeira linha é a **declaração**, com a versão e a codificação. Todo documento tem um único **elemento raiz**, aqui `<xmlcep>`, e os demais ficam aninhados dentro dele. Um elemento vazio pode ser escrito como `<unidade></unidade>` ou de forma abreviada, `<unidade/>`. Os elementos também podem ter **atributos**, como em `<tarefa id="42" status="DONE">`; a escolha entre atributo e elemento é de quem projeta o formato, e a regra mais comum é usar atributos para metadados curtos e elementos para os dados.

Cinco caracteres têm significado especial no XML e precisam ser trocados por **entidades** quando aparecem no conteúdo: `<` vira `<`, `>` vira `>`, `&` vira `&`, e as aspas viram `"` e `'`. Esquecer essa troca gera um XML inválido e, pior, abre espaço para ataques de injeção. Recursos como os ***namespaces*** (os `xmlns` vistos no envelope SOAP), que evitam conflito de nomes, e o **XML Schema** (XSD), que descreve e valida a estrutura, tornam o XML mais poderoso e mais formal que o JSON.

	JSON	XML
tamanho	menor	maior: cada nome aparece duas vezes
tipos	texto, número, booleano, nulo, objeto, lista	tudo é texto; os tipos vêm de um esquema (XSD)
listas	nativas, com `[ ]`	elementos repetidos
validação	JSON Schema (opcional)	XSD, muito usado
comentários	não aceita	aceita
uso típico	APIs REST e web	SOAP, documentos fiscais, sistemas legados

5.7 Respondendo em JSON ou em XML

Uma API pode oferecer mais de um formato e deixar o cliente escolher pelo cabeçalho `Accept`, a negociação de conteúdo apresentada no Capítulo 2. Para o TaskFlow, o JSON continua sendo o padrão, e quem enviar `Accept: application/xml` recebe XML. Primeiro, uma função que serializa um objeto simples:

```
// lib/xml.ts -- serializa um objeto simples em XML

// Os cinco caracteres que têm significado especial no XML
function escapar(valor: unknown): string {
  return String(valor)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll('"', "&quot;")
    .replaceAll("'", "&apos;");
}

export function paraXml(raiz: string, dados: object): string {
  const campos = Object.entries(dados).map(([nome, valor]) => {
    if (valor === null) return `  <${nome}/>`;
    const texto =
      valor instanceof Date ? valor.toISOString() : valor;
    return `  <${nome}>${escapar(texto)}</${nome}>`;
  });
  return [
    '<?xml version="1.0" encoding="UTF-8"?>',
    `<${raiz}>`,
    ...campos,
    `</${raiz}>`,
  ].join("\n");
}
```

A ordem das trocas importa: o `&` é trocado primeiro, porque as outras entidades também começam com `&`, e trocá-lo depois estragaria o `<` já escrito. O `Object.entries` transforma o objeto em uma lista de pares `[nome, valor]`, e cada par vira uma linha. As datas são convertidas para ISO 8601, como no JSON, e os valores nulos viram elementos vazios. Em seguida, no `GET` de uma tarefa, depois de conferir que ela existe:

```
// em app/api/tasks/[id]/route.ts, no fim do GET
const aceita = request.headers.get("accept") ?? "";
if (aceita.includes("application/xml")) {
  return new Response(paraXml("tarefa", tarefa), {
    headers: { "Content-Type": "application/xml; charset=utf-8" },
  });
}
return Response.json(tarefa);
```

Testando com uma tarefa cujo título tem caracteres especiais:

```
$ curl -i http://localhost:3000/api/tasks/cmvbb124... \
    -H "Accept: application/xml"
HTTP/1.1 200 OK
content-type: application/xml; charset=utf-8

<?xml version="1.0" encoding="UTF-8"?>
```

```
<tarefa>
  <id>cmvbb12470000icv9900ayko6</id>
  <title>Comprar &lt;papel&gt; &amp; caneta</title>
  <description/>
  <status>PENDING</status>
  <priority>MEDIUM</priority>
  <dueDate/>
  <createdAt>2026-10-16T18:31:55.639Z</createdAt>
  <updatedAt>2026-10-16T18:31:55.639Z</updatedAt>
</tarefa>
```

O título `Comprar <papel> & caneta` saiu com as entidades, e qualquer leitor de XML o reconstrói corretamente. Sem o `Accept`, a mesma URL devolve JSON. A função `paraXml` é propositalmente simples, para mostrar o que a serialização faz; ela não trata objetos aninhados nem listas. Em projetos reais, use uma biblioteca, como a `fast-xml-parser`, que serializa e desserializa XML completo. No navegador, o XML recebido é lido com a classe `DOMParser`, que já vem embutida.

5.8 Desserializar não é validar

Do lado de quem recebe, há um cuidado que o TypeScript esconde. O resultado de `await request.json()` tem o tipo `any`: o TypeScript não sabe nada sobre ele. É tentador escrever:

```
type NovaTarefa = { title: string; priority: "LOW" | "HIGH" };
const corpo = (await request.json()) as NovaTarefa;  // perigoso
```

O `as NovaTarefa` não confere nada: ele só diz ao TypeScript "confie em mim". Se o cliente enviar `{"title": 123, "priority": "URGENTE"}`, o código segue achando que `title` é um texto, e o erro aparece mais adiante, longe da causa, ou o dado errado vai parar no banco. Os tipos do TypeScript existem só enquanto o código é escrito e compilado; quando a requisição chega, eles já não existem. Todo dado que vem de fora da API precisa ser **validado em tempo de execução**, campo por campo. Fazer isso com vários `if`, como no `POST` do Capítulo 4, não escala. A solução é o assunto do próximo capítulo: descrever o formato esperado uma vez, com o Zod, e deixar que ele valide os dados e produza os tipos ao mesmo tempo.

Erros Comuns

- **Escrever JSON à mão com aspas simples ou vírgula sobrando.** O JSON só aceita aspas duplas e não tem vírgula depois do último item.
- **Usar `undefined` para "sem valor".** O campo desaparece da resposta. Use `null`.
- **Esperar uma data depois do `JSON.parse`.** O campo continua sendo texto; converta com `new Date(...)` quando precisar calcular.
- **Criar datas sem pensar no fuso.** `new Date("2026-12-01")` é meia-noite em UTC, e no Brasil aparece como o dia anterior. Guarde em UTC e converta só para exibir.
- **Aceitar datas no formato dd/mm/aaaa.** O JavaScript lê como mês/dia. Exija ISO 8601.

- **Devolver o registro inteiro do banco.** Campos internos, como `userId` ou a senha, vazam. Defina os campos públicos com `select`.
- **Montar XML sem escapar os caracteres especiais.** Um `&` ou um `<` no conteúdo invalida o documento e abre espaço para injeção.
- **Confiar no `as` do TypeScript para dados recebidos.** Ele não verifica nada em tempo de execução. Valide (Capítulo 6).

Capítulo 6 – Validação com Zod

O Capítulo 5 terminou com um alerta: os tipos do TypeScript desaparecem quando o programa roda, e o que chega em uma requisição pode ser qualquer coisa. Neste capítulo você vai entender por que toda API precisa validar os dados no servidor, conhecer o Zod, a biblioteca de validação mais usada no ecossistema TypeScript, e escrever os esquemas que descrevem uma tarefa do TaskFlow. No final, a criação de tarefas passa a aceitar todos os campos, recusa dados inválidos com mensagens claras para cada campo e ignora campos que o cliente não tem o direito de enviar.

6.1 Por que validar no servidor

Um formulário bem-feito valida os dados antes de enviar: campo obrigatório, tamanho máximo, e-mail no formato certo. Essa validação é ótima para a experiência de quem usa, porque o aviso aparece na hora. Mas ela **não protege a API**. Qualquer pessoa pode enviar uma requisição sem passar pelo formulário, com o `curl`, o REST Client ou um *script*; pode também abrir as ferramentas do navegador e alterar a página. Do ponto de vista da API, todo dado que chega é **não confiável**, venha de onde vier.

Sem validação no servidor, três coisas ruins acontecem. Dados errados vão parar no banco, como uma tarefa com título vazio ou uma data impossível. Erros que eram do cliente viram erros `500`, quando o banco ou o código quebram mais adiante, longe da causa, e a mensagem não ajuda ninguém a corrigir. E, pior, o cliente pode alterar o que não devia, como o dono de um registro, uma falha estudada na Seção 6.7. A regra é simples: **a validação no cliente é para conforto; a validação no servidor é obrigatória**.

6.2 O Zod

O **Zod** é uma biblioteca em que você descreve, em código, o formato que um dado deve ter, chamado de **esquema**. A partir do esquema, o Zod faz duas coisas: confere um valor qualquer em tempo de execução e fornece o tipo do TypeScript correspondente. Assim, a regra é escrita uma vez só e vale para a validação e para o editor. Instale a versão 4:

```
$ npm install zod@4
```

Um primeiro esquema e as duas formas de usá-lo:

```
import * as z from "zod";

const tarefa = z.object({
  title: z.string().trim().min(1).max(120),
  priority: z.enum(["LOW", "MEDIUM", "HIGH"]),
});

// parse: devolve o dado validado ou LANÇA um erro (ZodError)
const ok = tarefa.parse({
  title: "  Estudar Zod ",
  priority: "HIGH",
});
```

```
// { title: "Estudar Zod", priority: "HIGH" }

// safeParse: nunca lança; devolve um resultado para conferir
const r = tarefa.safeParse({ title: "", priority: "URGENTE" });
if (!r.success) {
  console.log(r.error.issues.length);   // 2
} else {
  console.log(r.data.title);
}
```

Repare que o `parse` devolveu o título sem os espaços: o `.trim()` não só confere, ele **transforma** o valor. O dado que sai do Zod é o dado limpo, pronto para usar. Em uma API, prefira o `safeParse`: dados inválidos são uma situação esperada, e não uma exceção, e o resultado com `success` obriga o código a tratar os dois casos. O TypeScript entende o `if`: dentro do `else`, ele sabe que `r.data` existe e tem o tipo do esquema.

6.3 Os blocos de um esquema

Os esquemas são montados combinando tipos básicos com regras encadeadas por pontos. Os mais usados em APIs:

Esquema	Aceita
`z.string().min(1).max(120)`	texto com 1 a 120 caracteres
`z.string().trim()`	texto, já sem os espaços das pontas
`z.email()`	texto no formato de e-mail
`z.number().int().positive()`	número inteiro maior que zero
`z.boolean()`	`true` ou `false`
`z.enum(["LOW", "HIGH"])`	um dos valores da lista
`z.iso.date()`	texto de data no formato `AAAA-MM-DD`
`z.array(z.string()).max(10)`	lista de até 10 textos
`z.object({ ... })`	objeto com os campos descritos
`.optional()`	o campo pode faltar (`undefined`)
`.nullable()`	o campo pode ser `null`
`.default(valor)`	se faltar, usa o valor indicado
`.transform(f)`	converte o valor aprovado com a função `f`
`.refine(f, ...)`	regra personalizada, escrita como função

A diferença entre `optional` e `nullable` importa em uma API. "Campo ausente" e "campo com `null`" significam coisas diferentes em uma edição: `{"title": "Novo"}` não mexe na data de vencimento, enquanto `{"dueDate": null}` pede para **apagar** a data. Por isso os campos que podem ser apagados aceitam os dois.

6.4 Mensagens de erro

Quando a validação falha, o Zod devolve uma lista de problemas (`issues`), cada um com o caminho do campo e uma mensagem. A função `z.flattenError` organiza essa lista por campo, o formato mais fácil para quem consome a API:

```
const r = tarefa.safeParse({ title: "", priority: "URGENTE" });
console.log(z.flattenError(r.error));
```

```
// {
//   formErrors: [],
//   fieldErrors: {
//     title: ["Too small: expected string to have
//              >=1 characters"],
//     priority: ["Invalid option: expected one of
//                 \"LOW\"|\"MEDIUM\"|\"HIGH\""]
//   }
// }
```

Os erros de um campo ficam em `fieldErrors`; os do objeto como um todo, em `formErrors`. As mensagens padrão estão em inglês. O Zod traz traduções, ligadas com `z.config(z.locales.pt())`, mas elas seguem o português de Portugal e são genéricas: "Demasiado pequeno: esperava que o texto tivesse >= 1 caracteres". Para uma API brasileira, o melhor é escrever as próprias mensagens, com a opção `error`:

```
z.string({ error: "O título precisa ser um texto." })
  .min(1, { error: "O título é obrigatório." })
```

A mensagem do `z.string(...)` vale quando o valor nem é um texto (um número, por exemplo, ou o campo ausente); a do `.min(...)` vale quando é um texto curto demais. Escreva mensagens que digam ao cliente o que fazer, e não só o que deu errado: "Use a data no formato AAAA-MM-DD" resolve o problema; "Data inválida", não.

6.5 Os esquemas do TaskFlow

Os esquemas da tarefa ficam em um arquivo próprio, separados das rotas:

```
// lib/esquemas/tarefas.ts -- o que a API aceita para uma tarefa
import * as z from "zod";
import {
  Priority,
  TaskStatus,
} from "@/app/generated/prisma/client";

// Os campos de uma tarefa, sem valores padrão: quem preenche o
// que faltar é o banco (os @default do schema.prisma).
const campos = z.object({
  title: z
    .string({ error: "O título precisa ser um texto." })
    .trim()
    .min(1, { error: "O título é obrigatório." })
    .max(120, {
      error: "O título pode ter até 120 caracteres.",
    }),
  description: z
    .string({ error: "A descrição precisa ser um texto." })
    .trim()
    .max(2000, {
      error: "A descrição pode ter até 2000 caracteres.",
```

```
    })
    .nullable(),
  status: z.enum(TaskStatus, {
    error: "status deve ser PENDING, IN_PROGRESS ou DONE.",
  }),
  priority: z.enum(Priority, {
    error: "priority deve ser LOW, MEDIUM ou HIGH.",
  }),
  dueDate: z.iso
    .date({ error: "Use a data no formato AAAA-MM-DD." })
    .transform((texto) => new Date(texto))
    .nullable(),
}, { error: "O corpo deve ser um objeto JSON." });

// POST: só o título é obrigatório
export const esquemaNovaTarefa = campos
  .partial()
  .required({ title: true });

// PATCH: tudo opcional, mas ao menos um campo
export const esquemaEdicaoTarefa = campos
  .partial()
  .refine((dados) => Object.keys(dados).length > 0, {
    error: "Envie ao menos um campo para alterar.",
  });

export type NovaTarefa = z.infer<typeof esquemaNovaTarefa>;
export type EdicaoTarefa = z.infer<typeof esquemaEdicaoTarefa>;
```

Os pontos principais:

- O `z.enum` aceita diretamente os objetos `TaskStatus` e `Priority` gerados pelo Prisma. Se um dia o esquema do banco ganhar uma prioridade nova, a validação a aceita sem nenhuma alteração aqui.
- O `dueDate` aceita só o formato ISO, o que barra o `01/12/2026` do Capítulo 5, e o `.transform` converte o texto aprovado no `Date` que o Prisma espera. O `.nullable()` vem depois, então `null` passa direto, sem conversão.
- O segundo parâmetro de `z.object` é a mensagem para quando o corpo nem é um objeto: `null` (JSON quebrado, graças ao `.catch` do Capítulo 4), uma lista ou um número.
- Os dois esquemas finais partem do mesmo conjunto de campos. O `.partial()` torna todos opcionais; o `.required({ title: true })` devolve a obrigatoriedade só ao título. Na edição, o `.refine` recusa um corpo vazio, que não alteraria nada.
- O `z.infer` extrai o tipo do TypeScript do esquema. Ninguém precisa escrever uma `interface` à parte, e o tipo nunca fica diferente da validação.

Atenção: valores padrão e partial

Seria natural escrever `priority: z.enum(Priority).default("MEDIUM")` no esquema de criação. Mas o `.partial()` mantém os valores padrão: um esquema de edição

derivado dele transforma `{}` em `{ priority: "MEDIUM" }`. Uma edição que mandasse só o título voltaria a prioridade para média sem ninguém pedir. Por isso os campos aqui não têm `.default()`: o banco já aplica os valores padrão na criação, e a edição só altera o que veio.

6.6 Validando nas rotas

Toda rota que recebe dados vai fazer o mesmo: validar e, se falhar, responder `400` com os erros por campo. Uma função auxiliar evita repetir isso:

```
// lib/validar.ts -- confere dados com um esquema do Zod
import * as z from "zod";

type Resultado<T> =
  | { dados: T; erro?: never }
  | { dados?: never; erro: Response };

export function validar<E extends z.ZodType>(
  esquema: E,
  entrada: unknown,
): Resultado<z.output<E>> {
  const resultado = esquema.safeParse(entrada);
  if (resultado.success) {
    return { dados: resultado.data };
  }

  const { formErrors, fieldErrors } =
    z.flattenError(resultado.error);
  return {
    erro: Response.json(
      {
        erro: formErrors[0] ?? "Dados inválidos.",
        campos: fieldErrors,
      },
      { status: 400 },
    ),
  };
}
```

A função é **genérica**: o `E` representa qualquer esquema, e `z.output<E>` é o tipo dos dados que ele produz (já transformados, como a data convertida). O tipo `Resultado` diz que ou há `dados`, ou há `erro`, nunca os dois; com isso, depois de testar o `erro`, o TypeScript sabe que `dados` existe. A rota de criação fica assim:

```
// em app/api/tasks/route.ts
import { validar } from "@/lib/validar";
import { esquemaNovaTarefa } from "@/lib/esquemas/tarefas";

// POST /api/tasks
export async function POST(request: NextRequest) {
```

```
  const usuario = await usuarioAtual();
  const corpo = await request.json().catch(() => null);

  const { dados, erro } = validar(esquemaNovaTarefa, corpo);
  if (erro) return erro;

  const tarefa = await db.task.create({
    data: { ...dados, userId: usuario.id },
    select: camposTarefa,
  });

  return Response.json(tarefa, {
    status: 201,
    headers: { Location: `/api/tasks/${tarefa.id}` },
  });
}
```

Os `if` do Capítulo 4 sumiram, e a rota ficou mais curta e mais completa: agora aceita descrição, situação, prioridade e data. Estes são resultados reais de requisições enviadas à rota:

```
POST {"title": " Entregar ", "priority": "HIGH",
      "dueDate": "2026-12-01"}
201 {"id": "cmvbb8py...", "title": "Entregar",
     "status": "PENDING", "priority": "HIGH",
     "dueDate": "2026-12-01T00:00:00.000Z", ...}

POST {"title": "", "priority": "URGENTE", "dueDate": "01/12/2026"}
400 {"erro": "Dados inválidos.",
     "campos": {"title": ["O título é obrigatório."],
                "priority": ["priority deve ser LOW, MEDIUM
                             ou HIGH."],
                "dueDate": ["Use a data no formato AAAA-MM-DD."]}}

POST {"title": 5}
400 {"erro": "Dados inválidos.",
     "campos": {"title": ["O título precisa ser um texto."]}}

POST {title          (JSON quebrado)
400 {"erro": "O corpo deve ser um objeto JSON.", "campos": {}}
```

Repare no segundo caso: **todos** os problemas vêm de uma vez, cada um no seu campo. O cliente corrige tudo em uma só tentativa, e um formulário pode mostrar cada mensagem embaixo do campo certo.

6.7 Campos que o cliente não pode enviar

Considere esta requisição, feita por alguém mal-intencionado:

```
POST /api/tasks
{"title": "Invasão", "userId": "id-de-outra-pessoa",
 "id": "id-escolhido", "createdAt": "2000-01-01"}
```

Se a rota passasse o corpo direto para o banco, com `db.task.create({ data: corpo })`, a tarefa seria criada na conta de outra pessoa, com o identificador e a data que o atacante escolheu. Essa falha se chama **atribuição em massa** (*mass assignment*): o cliente define campos que só o servidor deveria definir. Com o Zod, ela não acontece. O `z.object` **descarta** os campos que não estão no esquema, e a requisição acima cria uma tarefa comum, com `id` e `createdAt` gerados pelo banco e o `userId` de quem está logado, como foi conferido no teste. Por isso o `userId` é acrescentado na rota, depois da validação: `{ ...dados, userId: usuario.id }`.

Descartar os campos extras em silêncio é o comportamento padrão. Se preferir recusar a requisição quando vier um campo desconhecido, use `z.strictObject` no lugar de `z.object`: o Zod responde com um erro dizendo qual campo não é aceito. A escolha é de projeto; o importante é que **nenhum** campo chegue ao banco sem passar pelo esquema.

6.8 Validando a query string

Os parâmetros da URL também são dados de fora, e chegam sempre como texto. O Zod converte e valida com o `z.coerce`, que tenta transformar o valor no tipo pedido antes de conferir:

```
const paginacao = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
});

const params = request.nextUrl.searchParams;
paginacao.parse(Object.fromEntries(params));
// ?page=2          -> { page: 2, limit: 20 }
// ?limit=500       -> erro: o máximo é 100
// ?page=abc        -> erro: não é um número
```

O `Object.fromEntries` transforma os parâmetros da URL em um objeto comum, e o esquema faz o resto: converte `"2"` em 2, aplica os valores padrão e impõe o limite máximo combinado no Capítulo 2. Aqui o `.default()` é bem-vindo, porque a listagem sempre precisa de uma página e de um limite. O esquema completo de filtros da listagem, com situação, prioridade, busca e ordenação, entra no Capítulo 7, junto com a paginação.

Erros Comuns

- **Validar só no formulário.** Qualquer pessoa chama a API sem passar por ele. A validação no servidor é obrigatória.
- **Usar `parse` sem `try`.** Ele lança um erro que vira `500`. Nas rotas, use `safeParse` e responda `400`.
- **Usar o corpo original depois de validar.** Use o `dados` devolvido pelo Zod: ele está limpo, convertido e sem os campos extras. O `corpo` continua sujo.

- **Passar o corpo inteiro para o banco.** `create({ data: corpo })` permite atribuição em massa. Valide e acrescente os campos do servidor (como o `userId`) depois.
- **Derivar o esquema de edição de um esquema com `.default()`.** O `.partial()` mantém os padrões, e a edição altera campos que não vieram.
- **Confundir `optional` com `nullable`.** Ausente é "não mexa"; `null` é "apague".
- **Escrever a `interface` à mão ao lado do esquema.** Os dois ficam diferentes com o tempo. Use `z.infer`.
- **Mensagens que não ajudam.** "Inválido" não diz o que fazer. Diga o formato esperado.

Capítulo 7 – CRUD: manipulação de recursos

Até aqui, a API do TaskFlow cria, lista, busca e exclui tarefas, cada operação escrita direto na sua rota. Neste capítulo ela fica completa. Você vai organizar o acesso ao banco em uma camada de serviço, separada das rotas, implementar a edição com `PATCH`, transformar a listagem em uma consulta com filtros, busca por texto, ordenação e paginação, e entender as consultas do Prisma por trás de cada operação. No final, todos os *endpoints* de tarefas projetados no Capítulo 2 estarão funcionando e testados.

7.1 CRUD, HTTP e Prisma

CRUD é a sigla das quatro operações básicas sobre dados: *Create* (criar), *Read* (ler), *Update* (atualizar) e *Delete* (excluir). Em uma API REST, cada uma corresponde a um método HTTP, e no Prisma, a um grupo de funções:

Operação	HTTP	Status de sucesso	Prisma
Create	`POST` na coleção	`201` e `Location`	`create`
Read	`GET` na coleção ou no item	`200`	`findMany`, `findFirst`, `count`
Update	`PATCH` (ou `PUT`) no item	`200` e o recurso atualizado	`update`, `updateMany`
Delete	`DELETE` no item	`204`	`delete`, `deleteMany`

As funções terminadas em `Many` aceitam qualquer filtro e devolvem apenas quantos registros foram afetados. As outras trabalham com um único registro e devolvem o próprio registro; `update` e `delete` lançam um erro se ele não existir. Essa diferença decide qual usar em cada caso, como você vai ver na edição.

7.2 Uma camada de serviço

Nos capítulos anteriores, cada rota fazia tudo: lia a requisição, validava, consultava o banco e montava a resposta. Com o CRUD completo, as consultas crescem, e misturar HTTP com acesso a dados deixa as rotas longas e difíceis de testar. A solução comum é dividir o trabalho em **camadas**:

- a **rota** cuida do HTTP: lê a requisição, chama a validação e o serviço, e escolhe o status e o formato da resposta;
- o **serviço** cuida das regras e do banco: recebe dados já validados e devolve resultados, sem saber nada de requisições, cabeçalhos ou códigos de status.

Assim, a mesma função `listarTarefas` pode ser usada por uma rota da API, por uma página do Next.js ou por um teste automatizado (Capítulo 11). Os serviços das tarefas ficam em `lib/servicos/tarefas.ts`. As três operações mais simples:

```
// lib/servicos/tarefas.ts -- as operações sobre tarefas
import { db } from "@/lib/db";
import { Prisma } from "@/app/generated/prisma/client";
import { camposTarefa } from "@/lib/tarefas";
import type {
  EdicaoTarefa,
```

```
  Filtros,
  NovaTarefa,
} from "@/lib/esquemas/tarefas";

export async function criarTarefa(
  userId: string,
  dados: NovaTarefa,
) {
  return db.task.create({
    data: { ...dados, userId },
    select: camposTarefa,
  });
}

export async function buscarTarefa(userId: string, id: string) {
  return db.task.findFirst({
    where: { id, userId },
    select: camposTarefa,
  });
}

// Devolve true se excluiu, false se não havia o que excluir
export async function excluirTarefa(userId: string, id: string) {
  const { count } = await db.task.deleteMany({
    where: { id, userId },
  });
  return count > 0;
}
```

Todas as funções recebem o `userId` como primeiro parâmetro e o usam em toda consulta. É uma regra de projeto: no serviço de tarefas, não existe operação que ignore o dono. Assim, o isolamento entre as contas não depende de cada rota lembrar de filtrar. Repare também que `buscarTarefa` devolve `null` quando não acha, e `excluirTarefa` devolve `false`: decidir que isso vira um 404 é trabalho da rota.

7.3 Editando com PATCH

A edição recebe os campos validados pelo `esquemaEdicaoTarefa` do Capítulo 6 e altera só esses campos. O Prisma não mexe nos campos ausentes, e os que vieram com `null` (como `dueDate`) são apagados, exatamente a semântica do `PATCH`:

```
// Devolve a tarefa atualizada, ou null se ela não existir
// (ou for de outra pessoa)
export async function editarTarefa(
  userId: string,
  id: string,
  dados: EdicaoTarefa,
) {
  try {
    return await db.task.update({
      where: { id, userId },
```

```
      data: dados,
      select: camposTarefa,
    });
  } catch (erro) {
    if (
      erro instanceof Prisma.PrismaClientKnownRequestError &&
      erro.code === "P2025"
    ) {
      return null;
    }
    throw erro;
  }
}
```

O `update` precisa de um campo único no `where`, o `id`, mas aceita filtros extras, como o `userId`: a tarefa só é alterada se as duas condições valerem. Ele devolve a tarefa já atualizada em uma única consulta, o que o `updateMany` não faz (devolveria só a contagem, e seria preciso buscar a tarefa de novo). Em troca, quando nenhum registro atende ao filtro, o `update` lança um erro. O Prisma identifica os erros conhecidos por códigos, e `P2025` significa "registro não encontrado". O `catch` transforma só esse erro em `null`; qualquer outro, como o banco fora do ar, é relançado com `throw`, porque esconder um erro inesperado é pior que deixá-lo aparecer.

O `return await` dentro do `try` tem motivo: sem o `await`, a função devolveria a *Promise* antes de ela falhar, e o `catch` nunca seria executado.

7.4 A listagem completa

O Capítulo 2 combinou que a listagem aceita filtros, busca, ordenação e paginação. Primeiro, o esquema da *query string*, acrescentado a `lib/esquemas/tarefas.ts`:

```
// GET /api/tasks: os filtros da listagem, vindos da query string
const ordens = [
  "createdAt", "dueDate", "priority", "title",
] as const;

export const esquemaFiltros = z.object({
  status: z.enum(TaskStatus, {
    error: "status deve ser PENDING, IN_PROGRESS ou DONE.",
  }).optional(),
  priority: z.enum(Priority, {
    error: "priority deve ser LOW, MEDIUM ou HIGH.",
  }).optional(),
  search: z.string().trim().min(1).max(100).optional(),
  sort: z
    .enum([...ordens, ...ordens.map((o) => `-${o}` as const)], {
      error: "sort deve ser createdAt, dueDate, priority ou "
        + "title, com - na frente para a ordem decrescente.",
    })
    .default("-createdAt"),
  page: z.coerce
```

```
    .number({ error: "page deve ser um número." })
    .int({ error: "page deve ser um número inteiro." })
    .min(1, { error: "page começa em 1." })
    .default(1),
  limit: z.coerce
    .number({ error: "limit deve ser um número." })
    .int({ error: "limit deve ser um número inteiro." })
    .min(1, { error: "limit deve ser ao menos 1." })
    .max(100, { error: "limit pode ser no máximo 100." })
    .default(20),
});

export type Filtros = z.infer<typeof esquemaFiltros>;
```

O `sort` aceita só oito valores: os quatro campos, com ou sem o sinal de menos. A lista é montada a partir de `ordens`, e o `as const` faz o TypeScript conhecer cada valor exato, e não apenas "um texto". Limitar a ordenação a uma lista fechada é importante: aceitar qualquer nome de campo vindo da URL permitiria ordenar por campos internos ou provocar erros no banco. Agora, o serviço:

```
// Monta o orderBy do Prisma a partir de "dueDate" ou "-dueDate"
function ordenacao(sort: Filtros["sort"]) {
  const direcao = sort.startsWith("-") ? "desc" : "asc";
  const campo = sort.replace("-", "") as
    "createdAt" | "dueDate" | "priority" | "title";
  if (campo === "dueDate") {
    // tarefas sem data sempre no fim, nas duas direções
    return { dueDate: { sort: direcao, nulls: "last" } } as const;
  }
  return { [campo]: direcao };
}

export async function listarTarefas(userId: string, f: Filtros) {
  const where: Prisma.TaskWhereInput = {
    userId,
    status: f.status,
    priority: f.priority,
  };
  if (f.search) {
    where.OR = [
      { title: { contains: f.search, mode: "insensitive" } },
      {
        description: { contains: f.search, mode: "insensitive" },
      },
    ];
  }

  const [total, tarefas] = await db.$transaction([
    db.task.count({ where }),
    db.task.findMany({
      where,
```

```
      select: camposTarefa,
      orderBy: [ordenacao(f.sort), { id: "asc" }],
      skip: (f.page - 1) * f.limit,
      take: f.limit,
    }),
  ]);

  return {
    data: tarefas,
    meta: {
      page: f.page,
      limit: f.limit,
      total,
      totalPages: Math.ceil(total / f.limit),
    },
  };
}
```

Cada parte da consulta tem um papel:

- **Filtros.** No `where`, um campo com valor `undefined` é ignorado pelo Prisma. Por isso `status: f.status` só filtra quando o parâmetro veio na URL, sem precisar de `if`.
- **Busca.** O `OR` encontra a tarefa se o texto aparecer no título **ou** na descrição. O `contains` vira um `ILIKE '%texto%'` no PostgreSQL, e o `mode: "insensitive"` ignora maiúsculas e minúsculas, inclusive em letras acentuadas: `RELATÓRIO` encontra "Relatório". Mas ele **não** ignora acentos: `relatorio` não encontra "Relatório". Para isso, o PostgreSQL tem a extensão `unaccent`, ou se usa a busca textual completa (*full-text search*), fora do escopo desta apostila.
- **Ordenação.** A função `ordenacao` converte `"-dueDate"` no formato do Prisma. No PostgreSQL, os valores nulos ficam no fim na ordem crescente e no **início** na decrescente; o `nulls: "last"` mantém as tarefas sem data sempre no fim. O enum `Priority` é ordenado na ordem em que foi declarado no esquema, então `-priority` lista `HIGH` primeiro. O segundo critério, `{ id: "asc" }`, desempata tarefas iguais; sem ele, duas tarefas com a mesma prioridade podem trocar de lugar entre uma página e outra.
- **Paginação.** O `skip` pula as tarefas das páginas anteriores e o `take` limita a quantidade: na página 3 com 20 por página, pula 40 e pega 20.
- **Transação.** A contagem e a busca são duas consultas. O `$transaction` as executa juntas, em uma transação do banco, para que o `total` e a lista vejam os mesmos dados.

7.5 As rotas finais

Com o serviço pronto, as rotas ficam curtas: validar, chamar o serviço, responder.

```
// app/api/tasks/route.ts -- a coleção de tarefas
import type { NextRequest } from "next/server";
import { usuarioAtual } from "@/lib/usuario-atual";
import { validar } from "@/lib/validar";
```

```
import {
  esquemaFiltros,
  esquemaNovaTarefa,
} from "@/lib/esquemas/tarefas";
import {
  criarTarefa,
  listarTarefas,
} from "@/lib/servicos/tarefas";

// GET /api/tasks?status=&priority=&search=&sort=&page=&limit=
export async function GET(request: NextRequest) {
  const usuario = await usuarioAtual();
  const query = Object.fromEntries(request.nextUrl.searchParams);

  const { dados, erro } = validar(esquemaFiltros, query);
  if (erro) return erro;

  return Response.json(await listarTarefas(usuario.id, dados));
}

// POST /api/tasks
export async function POST(request: NextRequest) {
  const usuario = await usuarioAtual();
  const corpo = await request.json().catch(() => null);

  const { dados, erro } = validar(esquemaNovaTarefa, corpo);
  if (erro) return erro;

  const tarefa = await criarTarefa(usuario.id, dados);
  return Response.json(tarefa, {
    status: 201,
    headers: { Location: `/api/tasks/${tarefa.id}` },
  });
}
```

```
// app/api/tasks/[id]/route.ts -- uma tarefa
import type { NextRequest } from "next/server";
import { usuarioAtual } from "@/lib/usuario-atual";
import { validar } from "@/lib/validar";
import { esquemaEdicaoTarefa } from "@/lib/esquemas/tarefas";
import {
  buscarTarefa,
  editarTarefa,
  excluirTarefa,
} from "@/lib/servicos/tarefas";

type Contexto = RouteContext<"/api/tasks/[id]">;

const naoEncontrada = () =>
  Response.json(
    { erro: "Tarefa não encontrada." },
```

```
    { status: 404 },
  );

// GET /api/tasks/:id
export async function GET(
  request: NextRequest,
  ctx: Contexto,
) {
  const { id } = await ctx.params;
  const usuario = await usuarioAtual();

  const tarefa = await buscarTarefa(usuario.id, id);
  return tarefa ? Response.json(tarefa) : naoEncontrada();
}

// PATCH /api/tasks/:id
export async function PATCH(
  request: NextRequest,
  ctx: Contexto,
) {
  const { id } = await ctx.params;
  const usuario = await usuarioAtual();
  const corpo = await request.json().catch(() => null);

  const { dados, erro } = validar(esquemaEdicaoTarefa, corpo);
  if (erro) return erro;

  const tarefa = await editarTarefa(usuario.id, id, dados);
  return tarefa ? Response.json(tarefa) : naoEncontrada();
}

// DELETE /api/tasks/:id
export async function DELETE(
  request: NextRequest,
  ctx: Contexto,
) {
  const { id } = await ctx.params;
  const usuario = await usuarioAtual();

  const excluiu = await excluirTarefa(usuario.id, id);
  return excluiu
    ? new Response(null, { status: 204 })
    : naoEncontrada();
}
```

O tipo `Contexto` e a função `naoEncontrada` evitam repetir o mesmo trecho nas três funções. A resposta em XML do Capítulo 5, que serviu para mostrar a negociação de conteúdo, foi retirada para simplificar; se quiser mantê-la, ela continua funcionando no `GET`.

7.6 Testando o CRUD

Os resultados abaixo vieram da API em funcionamento, com a usuária de teste e mais algumas tarefas. Para testar o isolamento, foi criada no banco uma segunda pessoa com uma tarefa própria.

```
GET /api/tasks
200 meta: {"page":1, "limit":20, "total":13, "totalPages":1}

GET /api/tasks?search=RELAT
200 títulos: ["Relatório final", "Revisar o relatório"]

GET /api/tasks?sort=-priority&limit=4
200 prioridades: HIGH, HIGH, HIGH, HIGH

GET /api/tasks?sort=dueDate
200 datas: 2026-11-05, 2026-11-10, 2026-12-01, 2026-12-20, (sem)

GET /api/tasks?limit=3&page=2
200 3 tarefas; meta: {"page":2, "limit":3, "total":13,
                      "totalPages":5}

GET /api/tasks?limit=500&sort=nome&page=0
400 {"erro":"Dados inválidos.","campos":{
      "sort":["sort deve ser createdAt, dueDate, ..."],
      "page":["page começa em 1."],
      "limit":["limit pode ser no máximo 100."]}}

PATCH /api/tasks/<id>  {"status":"DONE"}
200 a tarefa com "status":"DONE" e o updatedAt novo

PATCH /api/tasks/<id>  {}
400 {"erro":"Envie ao menos um campo para alterar.","campos":{}}

PATCH /api/tasks/<id>
      {"dueDate":null,"title":"Comprar café e leite"}
200 a tarefa sem data e com o título novo

PATCH, GET e DELETE em /api/tasks/<id da outra pessoa>
404 {"erro":"Tarefa não encontrada."}
```

Repare em dois resultados. Uma página além do fim, como `?page=99`, não é erro: devolve `data` vazio e o `meta` correto, e o cliente percebe pelo `totalPages` que passou do fim. E a tarefa da outra pessoa recebe exatamente a mesma resposta que uma tarefa inexistente, nas três operações.

7.7 E o PUT?

A API do TaskFlow não implementa `PUT`. O `PATCH` cobre todas as edições, e oferecer os dois obrigaria a manter duas regras de validação para a mesma operação. Se a rota recebe um `PUT`, o Next.js responde `405`, como visto no Capítulo 4. Isso é comum em APIs reais: implemente os métodos que fazem sentido para os seus clientes, e não todos os possíveis.

Um último cuidado sobre edições: se duas pessoas (ou duas abas do navegador) editam a mesma tarefa quase ao mesmo tempo, a última gravação vence, e a primeira se perde sem aviso. Para o TaskFlow, em que cada pessoa edita só as próprias tarefas, o risco é pequeno. Sistemas em que muitos editam o mesmo registro, como um estoque, usam o **controle de concorrência otimista**: o cliente envia junto a versão que leu (por exemplo, o `updatedAt`), e o servidor recusa a gravação com `409 Conflict` se o registro mudou nesse meio-tempo.

Erros Comuns

- **Esquecer o dono em uma consulta.** Uma única consulta sem `userId` expõe dados de todos. Centralize as consultas em um serviço que sempre recebe o dono.
- **Usar `update` sem tratar o erro `P2025`.** A tarefa inexistente vira `500`. Trate o `P2025` e responda `404`.
- **Esconder todos os erros no `catch`.** Trate só o erro esperado e relance os outros com `throw`.
- **`return` sem `await` dentro do `try`.** A *Promise* sai da função antes de falhar, e o `catch` não a vê.
- **Aceitar qualquer campo em `sort`.** Limite a ordenação a uma lista fechada.
- **Paginar sem desempate.** Com valores iguais no campo ordenado, a ordem entre eles não é garantida, e itens se repetem ou somem entre as páginas. Acrescente o `id` como segundo critério.
- **Contar e buscar em momentos diferentes.** Use `$transaction` para que o total e a lista sejam consistentes.
- **Esperar que a busca ignore acentos.** O `mode: "insensitive"` ignora maiúsculas, não acentos.

Capítulo 8 – Erros e códigos HTTP

Até aqui, cada parte da API do TaskFlow respondeu aos erros do seu jeito: a validação devolve `{erro, campos}`, as rotas devolvem `{erro}`, uma falha inesperada devolve um `500` sem corpo e um endereço inexistente devolve uma página HTML. Neste capítulo você vai aprender a escolher o código de status certo para cada situação, adotar um formato único para todos os erros, o *Problem Details* da RFC 9457, e criar um tratador central, que transforma qualquer erro lançado em qualquer parte do código em uma resposta bem formada. No final, o `500` passa a ser seguro e rastreável, e a API fica pronta para a autenticação do Capítulo 9.

8.1 O erro também é uma resposta

Uma resposta de erro tem dois leitores. O primeiro é um **programa**, o cliente da API, que precisa decidir o que fazer: mostrar as mensagens no formulário, pedir um novo *login*, tentar de novo mais tarde. O segundo é uma **pessoa**, quem desenvolve o cliente, que precisa entender o que deu errado e como corrigir. Um bom erro atende os dois: tem o **código de status certo**, para o programa; um **formato previsível**, para que o mesmo código leia qualquer erro da API; e uma **mensagem útil**, para a pessoa, sem revelar nada que não deva.

O Capítulo 1 mostrou um exemplo que falha no primeiro ponto. O ViaCEP, consultado com um CEP que não existe, responde `200` com o corpo `{"erro": "true"}`. Para o HTTP, essa requisição deu certo, e quem olha só o status é enganado:

- o cliente não pode confiar no `resposta.ok` do `fetch`, que vale `true` para todo `2xx`: precisa conhecer e conferir o campo `erro`, uma regra particular dessa API;
- bibliotecas como o Axios, que tratam os códigos `4xx` e `5xx` como falha automaticamente, tratam a resposta como sucesso;
- ferramentas de monitoramento, que contam quantas respostas de erro a API dá por minuto, não enxergam o problema.

Com `404 Not Found`, o recurso "endereço do CEP 99999-999" simplesmente não existe, e todas as ferramentas entendem isso sem ler a documentação. É a ideia do nível 2 do modelo de maturidade (Capítulo 2): usar os códigos de status com o seu significado.

8.2 Escolhendo o código de status

A tabela do Capítulo 2 apresentou os códigos mais comuns. Esta é a versão completa para os erros, com um exemplo do TaskFlow para cada um:

Código	Quando usar	No TaskFlow
`400 Bad Request`	dados malformados ou inválidos	título vazio, JSON quebrado, `?page=0`
`401 Unauthorized`	falta autenticação, ou ela é inválida	requisição sem *token* (Capítulo 9)
`403 Forbidden`	a pessoa é conhecida, mas não tem permissão	uma área só para administradores
`404 Not Found`	o recurso não existe, ou a pessoa não pode saber que ele existe	tarefa inexistente ou de outra pessoa
`405 Method Not Allowed`	a URL existe, mas não aceita o método	`PUT /api/tasks`
`409 Conflict`	conflito com o estado atual do recurso	e-mail já cadastrado (Capítulo 9)
`415 Unsupported Media Type`	o corpo veio em um formato que a rota não aceita	XML onde só se aceita JSON
`422 Unprocessable Content`	dados bem formados, mas que violam uma regra	alternativa ao `400`, discutida abaixo
`429 Too Many Requests`	a pessoa passou do limite de requisições	limite de tentativas de *login* (Capítulo 12)
`500 Internal Server Error`	falha inesperada no servidor	banco de dados fora do ar
`503 Service Unavailable`	o serviço está fora do ar por um tempo	manutenção programada

Quando mais de um código parece servir, a ordem em que a API faz as verificações resolve a dúvida. Uma sequência comum, e a que o TaskFlow segue, é esta:

1. a URL e o método existem? Se não, `404` ou `405`;
2. quem está pedindo? Sem autenticação válida, `401`;
3. os dados estão no formato certo? Se não, `400`;
4. o recurso existe para essa pessoa? Se não, `404`;
5. ela pode fazer essa operação? Se não, `403`;
6. a operação conflita com o estado atual? Se sim, `409`;
7. deu tudo certo: `200`, `201` ou `204`.

A autenticação vem antes de tudo por segurança: quem não se identificou não deve aprender nada sobre a API, nem as regras de validação, nem quais recursos existem. Alguns pares de códigos merecem atenção:

- **`400` ou `422`?** A RFC 9110, que define o HTTP, descreve o `422` para dados que estão no formato certo, mas violam uma regra, como uma data de vencimento no passado. Por isso, algumas APIs, como a do GitHub e as feitas com FastAPI, usam `400` para o JSON quebrado e `422` para a validação; outras usam `400` para tudo. As duas escolhas são aceitas. O TaskFlow usa `400` para todos os dados inválidos, como foi combinado no Capítulo 2. O que não pode é misturar: escolha uma regra e registre-a na documentação (Capítulo 10).
- **`401` ou `403`?** Apesar do nome, `Unauthorized`, o `401` quer dizer "não autenticado": a API não sabe quem é você. O `403` quer dizer "sei quem é você, e você não pode". Repetir a requisição com outro *token* pode resolver um `401`; um `403`, não.

- **403 ou 404?** Como combinado no Capítulo 2, a tarefa de outra pessoa recebe `404`, porque um `403` confirmaria que ela existe. O `403` fica para quando a existência do recurso não é segredo, mas a operação é proibida.
- **4xx ou 5xx?** Um `5xx` diz "a culpa é do servidor". Se um dado enviado pelo cliente provoca um `500`, a API tem um defeito: faltou validar esse dado e responder `400`. Todo `500` que aparece nos testes é um defeito a corrigir.

8.3 Um formato para todos os erros

Hoje, o cliente do TaskFlow precisaria de quatro formas de ler um erro: `{erro, campos}` da validação, `{erro}` das rotas, um corpo vazio no `500` e uma página HTML para os endereços que não existem. Além disso, os nomes `erro` e `campos` quebram a convenção do Capítulo 5, de campos em inglês. É hora de padronizar.

Em vez de inventar um formato, a API pode adotar um padrão. A **RFC 9457**, *Problem Details for HTTP APIs*, publicada em 2023 para substituir a RFC 7807, define um formato de erro em JSON, com um tipo de mídia próprio, `application/problem+json`. Os campos previstos são estes:

Campo	Conteúdo
`type`	uma URI que identifica o tipo de problema; se faltar, vale `"about:blank"`, "nenhum tipo além do próprio status"
`title`	um resumo curto do tipo de problema, que não muda de uma ocorrência para outra
`status`	o código de status HTTP, repetido no corpo
`detail`	a explicação desta ocorrência, escrita para pessoas
`instance`	uma URI que identifica esta ocorrência específica

Todos são opcionais, e a API pode acrescentar campos próprios, as **extensões**. A mais comum é uma lista de erros por campo, que o TaskFlow vai chamar de `errors`. Um erro de validação fica assim:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "title": "Requisição inválida",
  "status": 400,
  "detail": "Um ou mais campos estão inválidos.",
  "errors": {
    "title": ["O título é obrigatório."],
    "priority": ["priority deve ser LOW, MEDIUM ou HIGH."]
  }
}
```

O TaskFlow não usa o campo `type`. Nesse caso, a RFC pede que o `title` seja o nome do próprio status, e permite traduzi-lo: por isso o título é sempre o mesmo para cada código, como "Não encontrado" para o `404`, e a explicação específica vai no `detail`. APIs maiores usam o `type` para distinguir problemas diferentes com o mesmo status, como `https://api.exemplo.com/erros/saldo-insuficiente` e `.../cartao-recusado`, ambos `400`; a URI pode apontar para uma página que explica o problema.

Uma regra vale para qualquer formato: o programa cliente decide pelo `status` (e pelo `type`, se houver), **nunca** pelo texto do `detail`. Mensagens mudam, são corrigidas e traduzidas; um cliente que compara textos quebra na primeira revisão.

8.4 O módulo de erros

Tudo o que diz respeito a erros fica em um arquivo, `lib/erros.ts`. A primeira parte monta a resposta no formato da RFC 9457:

```
// lib/erros.ts -- as respostas de erro da API, no formato
// Problem Details (RFC 9457)
import type { NextRequest } from "next/server";

const titulos: Record<number, string> = {
  400: "Requisição inválida",
  401: "Não autenticado",
  403: "Acesso negado",
  404: "Não encontrado",
  409: "Conflito",
  500: "Erro interno",
};

type Campos = Record<string, string[] | undefined>;

// Monta a resposta de erro; os extras (errors, instance) são
// acrescentados ao corpo quando existem
export function problema(
  status: number,
  detail: string,
  extras: { errors?: Campos; instance?: string } = {},
) {
  return Response.json(
    {
      title: titulos[status] ?? "Erro",
      status,
      detail,
      ...extras,
    },
    {
      status,
      headers: { "Content-Type": "application/problem+json" },
    },
  );
}
```

O `Response.json` só acrescenta o `Content-Type: application/json` quando ele não foi informado; aqui, o cabeçalho passado em `headers` prevalece, e a resposta sai como `application/problem+json`. O tipo `Record<number, string>` descreve um objeto cujas chaves são números e cujos valores são textos. Para um status fora da lista, o `??` usa o título genérico "Erro". Com essa função, uma rota responde um erro em uma linha: `return problema(404, "Tarefa não encontrada.")`.

Mas há erros que não nascem na rota. A validação, uma função auxiliar, descobre

que os dados são inválidos; no Capítulo 9, a função `usuarioAtual` vai descobrir que o *token* é inválido. Fazer essas funções devolverem uma resposta, e cada rota conferir se recebeu uma, enche o código de `if`. A solução é **lançar** o erro, com `throw`, e tratá-lo em um único lugar. Para isso, um tipo de erro que já sabe qual resposta virar:

```
// Um erro que já sabe qual resposta virar. Qualquer parte do
// código pode lançá-lo; o tratador central (abaixo) responde.
export class ErroApi extends Error {
  status: number;
  campos?: Campos;

  constructor(status: number, mensagem: string, campos?: Campos) {
    super(mensagem);
    this.status = status;
    this.campos = campos;
  }
}
```

A classe `ErroApi` herda de `Error` (`extends`), então tem tudo o que um erro comum tem, como a mensagem e o registro de onde foi lançado, e acrescenta o status e os erros por campo. O `super(mensagem)` chama o construtor de `Error`, que guarda a mensagem em `this.message`. Qualquer função pode agora fazer `throw new ErroApi(409, "Este e-mail já está em uso.")`, e o erro sobe pelas funções que a chamaram até que alguém o trate.

Atenção: erros de HTTP e camadas

O `ErroApi` carrega um status HTTP, e o Capítulo 7 combinou que os serviços não sabem nada de HTTP. Em projetos maiores, os serviços lançam erros do próprio domínio, como `EmailEmUso` ou `SaldoInsuficiente`, e só o tratador central os traduz em status. No TaskFlow, que é pequeno, o `ErroApi` será usado pela validação e pela autenticação, que já são partes ligadas ao HTTP, e os serviços continuam devolvendo `null` ou `false`.

Por fim, o tratador. A função `respostaDeErro` recebe qualquer coisa que tenha sido lançada e escolhe a resposta; a `comErros` envolve uma função de rota, para que nada do que ela lançar escape sem tratamento:

```
export function respostaDeErro(erro: unknown) {
  if (erro instanceof ErroApi) {
    return problema(erro.status, erro.message, {
      errors: erro.campos,
    });
  }

  // Erro inesperado: o detalhe fica só no log do servidor
  const id = crypto.randomUUID();
  console.error(`[erro ${id}]`, erro);
  return problema(
    500,
    "Algo deu errado no servidor. Tente de novo mais tarde.",
```

```
    { instance: `urn:uuid:${id}` },
  );
}

// Envolve uma função de rota: o que ela lançar vira resposta
export function comErros<C>(
  rota: (request: NextRequest, ctx: C) => Promise<Response>,
) {
  return async (request: NextRequest, ctx: C) => {
    try {
      return await rota(request, ctx);
    } catch (erro) {
      return respostaDeErro(erro);
    }
  };
}
```

O parâmetro de `respostaDeErro` tem o tipo `unknown`, porque em JavaScript qualquer valor pode ser lançado, até um texto ou um número. O `instanceof` confere se o erro é um `ErroApi`: se for, ele é um erro **esperado**, e vira a resposta que ele mesmo descreve. Qualquer outra coisa é um erro **inesperado**, assunto da próxima seção.

A `comErros` é uma função que recebe uma função e devolve outra, com o mesmo formato, mas protegida por um `try`/`catch`. O `<C>` a torna genérica: `C` é o tipo do contexto da rota, que muda de rota para rota, e o TypeScript o descobre sozinho a partir da função recebida. O `return await` dentro do `try` tem o mesmo motivo do Capítulo 7: sem o `await`, a *Promise* sairia da função antes de falhar, e o `catch` não a veria.

8.5 O erro inesperado

Erros inesperados acontecem em qualquer API: o banco sai do ar, um serviço externo demora demais, um caso que ninguém previu chega pela primeira vez. Antes deste capítulo, o TaskFlow deixava o Next.js cuidar deles. Estes resultados foram obtidos com a versão do Capítulo 7 e uma rota de teste que lança um erro:

Situação	Resposta do Next.js
erro lançado dentro da rota	`500`, com o corpo vazio
`GET /api/naoexiste`	`404`, com uma página HTML de 7 KB
`PUT /api/tasks`	`405`, com o corpo vazio

O Next.js acerta no essencial: a mensagem do erro vai só para o terminal do servidor, e não para o cliente. Isso importa, porque a mensagem de um erro inesperado costuma conter o que ninguém de fora deve ver: o SQL que falhou, nomes de tabelas, caminhos de arquivos, versões de bibliotecas, às vezes até uma *connection string*. Essas informações ajudam um atacante a planejar o próximo passo, e por isso a regra é absoluta: **a mensagem e a pilha de chamadas de um erro inesperado nunca vão para a resposta**.

Mas um corpo vazio também não ajuda. Quem usa a API não sabe o que aconteceu, e quem a mantém não tem como achar, entre milhares de linhas de *log*, o erro de que a pessoa está falando. A `respostaDeErro` resolve as duas coisas com um **identificador**

de ocorrência. O `crypto.randomUUID()`, embutido no Node.js e nos navegadores, gera um código aleatório, praticamente impossível de repetir; o mesmo código vai para o *log*, junto com o erro completo, e para a resposta, no campo `instance`, no formato de URI `urn:uuid:...`. Para testar, o banco de dados foi desligado com o servidor no ar:

```
GET /api/tasks           (com o banco desligado)
500 Content-Type: application/problem+json
    {"title":"Erro interno","status":500,
     "detail":"Algo deu errado no servidor. Tente de novo
               mais tarde.",
     "instance":"urn:uuid:da4a7691-89c4-4506-bd99-2691e554e182"}

No terminal do servidor:
[erro da4a7691-89c4-4506-bd99-2691e554e182] Error
  [PrismaClientKnownRequestError]:
Invalid `prisma.user.findUniqueOrThrow()` invocation:
  ...
  code: 'ECONNREFUSED',
```

O cliente recebe uma resposta no formato de sempre, sem nenhum detalhe interno. Se a pessoa entrar em contato com o suporte e informar o código, quem mantém a API procura `da4a7691` no *log* e encontra a causa: o banco recusou a conexão (`ECONNREFUSED`). Quando o banco voltou, a API voltou a responder normalmente, sem reiniciar. Na Vercel (Capítulo 12), tudo o que o `console.error` escreve aparece no painel de *logs*; em projetos maiores, serviços como o Sentry recebem esses erros e avisam a equipe.

8.6 A validação lança o erro

Com o tratador central, a função `validar` do Capítulo 6 fica mais simples. Em vez de devolver um objeto com `dados` ou `erro`, ela devolve os dados validados ou lança um `ErroApi`:

```
// lib/validar.ts -- confere dados com um esquema do Zod
import * as z from "zod";
import { ErroApi } from "@/lib/erros";

// Devolve os dados validados ou lança um ErroApi com o 400
export function validar<E extends z.ZodType>(
  esquema: E,
  entrada: unknown,
): z.output<E> {
  const resultado = esquema.safeParse(entrada);
  if (resultado.success) return resultado.data;

  const { formErrors, fieldErrors } =
    z.flattenError(resultado.error);
  throw new ErroApi(
    400,
    formErrors[0] ?? "Um ou mais campos estão inválidos.",
    fieldErrors,
```

```
  );
}
```

O tipo `Resultado` do Capítulo 6 desapareceu: o tipo de retorno agora é simplesmente `z.output<E>`, os dados validados. Se a validação falha, a função não retorna; o `throw` interrompe a rota que a chamou, e o tratador responde o `400`. O `detail` é o erro do objeto como um todo, quando existe ("O corpo deve ser um objeto JSON.", "Envie ao menos um campo para alterar."), ou uma frase geral, quando os problemas estão nos campos. O `errors` está sempre presente nas respostas `400` da validação, mesmo que vazio; assim, o cliente pode lê-lo sem conferir antes se ele existe.

8.7 As rotas com o tratador

As rotas de tarefas passam a ser envolvidas pela `comErros`. A coleção:

```
// app/api/tasks/route.ts -- a coleção de tarefas
import type { NextRequest } from "next/server";
import { comErros } from "@/lib/erros";
import { usuarioAtual } from "@/lib/usuario-atual";
import { validar } from "@/lib/validar";
import {
  esquemaFiltros,
  esquemaNovaTarefa,
} from "@/lib/esquemas/tarefas";
import {
  criarTarefa,
  listarTarefas,
} from "@/lib/servicos/tarefas";

// GET /api/tasks?status=&priority=&search=&sort=&page=&limit=
export const GET = comErros(async (request: NextRequest) => {
  const usuario = await usuarioAtual();
  const query = Object.fromEntries(request.nextUrl.searchParams);

  const filtros = validar(esquemaFiltros, query);
  return Response.json(await listarTarefas(usuario.id, filtros));
});

// POST /api/tasks
export const POST = comErros(async (request: NextRequest) => {
  const usuario = await usuarioAtual();
  const corpo = await request.json().catch(() => null);

  const dados = validar(esquemaNovaTarefa, corpo);
  const tarefa = await criarTarefa(usuario.id, dados);
  return Response.json(tarefa, {
    status: 201,
    headers: { Location: `/api/tasks/${tarefa.id}` },
  });
});
```

As rotas deixaram de ser declaradas com `export async function GET` e passaram a

ser constantes, `export const GET = comErros(...)`. Para o Next.js não há diferença: ele procura no arquivo algo exportado com o nome do método e que seja uma função, e o resultado de `comErros` é uma função. O `if (erro) return erro` do Capítulo 7 sumiu das duas rotas. Agora, uma tarefa:

```
// app/api/tasks/[id]/route.ts -- uma tarefa
import type { NextRequest } from "next/server";
import { comErros, problema } from "@/lib/erros";
import { usuarioAtual } from "@/lib/usuario-atual";
import { validar } from "@/lib/validar";
import { esquemaEdicaoTarefa } from "@/lib/esquemas/tarefas";
import {
  buscarTarefa,
  editarTarefa,
  excluirTarefa,
} from "@/lib/servicos/tarefas";

type Contexto = RouteContext<"/api/tasks/[id]">;

const naoEncontrada = () =>
  problema(404, "Tarefa não encontrada.");

// GET /api/tasks/:id
export const GET = comErros(
  async (request: NextRequest, ctx: Contexto) => {
    const { id } = await ctx.params;
    const usuario = await usuarioAtual();

    const tarefa = await buscarTarefa(usuario.id, id);
    return tarefa ? Response.json(tarefa) : naoEncontrada();
  },
);

// PATCH /api/tasks/:id
export const PATCH = comErros(
  async (request: NextRequest, ctx: Contexto) => {
    const { id } = await ctx.params;
    const usuario = await usuarioAtual();
    const corpo = await request.json().catch(() => null);

    const dados = validar(esquemaEdicaoTarefa, corpo);
    const tarefa = await editarTarefa(usuario.id, id, dados);
    return tarefa ? Response.json(tarefa) : naoEncontrada();
  },
);

// DELETE /api/tasks/:id
export const DELETE = comErros(
  async (request: NextRequest, ctx: Contexto) => {
    const { id } = await ctx.params;
    const usuario = await usuarioAtual();
```

```
    const excluiu = await excluirTarefa(usuario.id, id);
    return excluiu
      ? new Response(null, { status: 204 })
      : naoEncontrada();
  },
);
```

Repare que a tarefa não encontrada continua sendo **devolvida** com `return`, e não lançada. A regra prática é esta: dentro da própria rota, onde a resposta está à mão, `return problema(...)` deixa o caminho do código visível; o `throw new ErroApi(...)` é para as funções auxiliares, que estão longe da resposta e não devem obrigar a rota a conferir o que elas devolveram.

O tratador também prepara o próximo capítulo. A função `usuarioAtual`, hoje provisória, vai ler o *token* da requisição e, se ele faltar ou for inválido, lançar um `ErroApi` com o `401`. Como todas as rotas já passam pela `comErros`, esse `401` vira a resposta certa sem que nenhuma rota seja alterada.

8.8 Endereços que não existem

Falta a página HTML do `404`. Um cliente que erra o endereço, como `/api/task` em vez de `/api/tasks`, recebe 7 KB de HTML, e o `resposta.json()` dele falha com um erro confuso. A solução usa uma convenção de pastas do Capítulo 4, o segmento `[...caminho]`, que atende a um ou mais segmentos quaisquer:

```
// app/api/[...rota]/route.ts -- endereços que não existem
import { problema } from "@/lib/erros";

async function naoExiste() {
  return problema(404, "Este endereço não existe na API.");
}

export const GET = naoExiste;
export const POST = naoExiste;
export const PUT = naoExiste;
export const PATCH = naoExiste;
export const DELETE = naoExiste;
```

A pasta `app/api/[...rota]` combina com qualquer endereço que comece com `/api/`. Como vence sempre a rota mais específica, `/api/tasks` e `/api/tasks/:id` continuam indo para as suas pastas, e só o que não combina com nenhuma outra cai aqui, como `/api/naoexiste` ou `/api/tasks/a/b`. As URLs fora de `/api`, que não fazem parte da API, continuam com o `404` padrão do Next.js.

O `405` continua sendo respondido pelo Next.js, sem corpo. A RFC 9110 pede que ele traga o cabeçalho `Allow`, com os métodos aceitos; na versão testada, o Next.js não o envia no `405`, mas o envia na resposta a `OPTIONS`, onde o cliente pode consultá-lo. O projeto fica assim:

```
app/api/
  [...rota]/route.ts    404 em JSON para o que não existe
  health/route.ts
  tasks/route.ts        GET, POST (com comErros)
  tasks/[id]/route.ts   GET, PATCH, DELETE (com comErros)
lib/
  erros.ts              problema, ErroApi, comErros
  validar.ts            agora lança ErroApi
```

8.9 Testando os erros

Estes são resultados reais da API com o código deste capítulo. Os cabeçalhos repetidos foram omitidos: todas as respostas de erro trazem `Content-Type: application/problem+json`.

```
GET /api/tasks?limit=500&sort=nome&page=0
400 {"title":"Requisição inválida","status":400,
     "detail":"Um ou mais campos estão inválidos.",
     "errors":{"sort":["sort deve ser createdAt, ..."],
               "page":["page começa em 1."],
               "limit":["limit pode ser no máximo 100."]}}

POST /api/tasks  {title          (JSON quebrado)
400 {"title":"Requisição inválida","status":400,
     "detail":"O corpo deve ser um objeto JSON.","errors":{}}

PATCH /api/tasks/<id>  {}
400 {"title":"Requisição inválida","status":400,
     "detail":"Envie ao menos um campo para alterar.",
     "errors":{}}

GET, PATCH e DELETE em /api/tasks/naoexiste
404 {"title":"Não encontrado","status":404,
     "detail":"Tarefa não encontrada."}

GET /api/naoexiste  e  POST /api/tasks/a/b
404 {"title":"Não encontrado","status":404,
     "detail":"Este endereço não existe na API."}

PUT /api/tasks
405 (sem corpo)

OPTIONS /api/tasks
204 allow: GET, HEAD, OPTIONS, POST
```

As respostas de sucesso não mudaram: a criação continua respondendo `201` com `Content-Type: application/json`, a tarefa e o `Location`. Agora, qualquer erro da API, de qualquer rota, pode ser lido pelo mesmo código.

8.10 Do lado do cliente

Um detalhe do `fetch` pega muita gente: ele **não** lança um erro quando a resposta é `4xx` ou `5xx`. Para o `fetch`, receber um `404` é sucesso, porque o servidor respondeu; ele só lança um erro quando não há resposta nenhuma, como na falta de internet. Quem confere o resultado é o cliente, com a propriedade `ok`, que vale `true` para os status de 200 a 299:

```
async function criarTarefa(dados) {
  const resposta = await fetch("/api/tasks", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados),
  });

  if (!resposta.ok) {
    const problema = await resposta.json();
    console.log(resposta.status, problema.detail);
    console.log(problema.errors?.title);
    return null;
  }
  return resposta.json();
}

await criarTarefa({ title: "" });
// 400 Um ou mais campos estão inválidos.
// [ 'O título é obrigatório.' ]
```

Como todos os erros do TaskFlow têm o mesmo formato, esse único `if` serve para qualquer chamada. Uma interface pode mostrar o `detail` em um aviso e cada mensagem de `errors` embaixo do campo correspondente do formulário; e pode reagir ao `status`, levando à tela de *login* no `401`, por exemplo. Se a API estiver atrás de outros servidores, como um balanceador de carga, um erro pode chegar de um deles, em HTML; um cliente cuidadoso confere o cabeçalho `Content-Type` antes de chamar o `json()`.

Erros Comuns

- **Responder 200 com um erro no corpo.** Clientes, bibliotecas e ferramentas de monitoramento entendem como sucesso. Use o código de status do erro.
- **Mandar a mensagem do erro inesperado para o cliente.** `erro.message` e a pilha de chamadas revelam detalhes internos. Registre-os no *log* e responda uma mensagem genérica, com um identificador.
- **Aceitar um 500 provocado por dados do cliente.** Se uma entrada inválida derruba a rota, faltou validação. Responda `400`.
- **Usar 403 para o recurso de outra pessoa.** Ele confirma que o recurso existe. Responda `404`.
- **Confundir 401 e 403.** 401: "não sei quem é você". 403: "sei quem é você, e você não pode".
- **Um formato de erro por rota.** O cliente precisa de um único jeito de ler os erros. Centralize a montagem da resposta em uma função.

- **Cliente que decide pelo texto do erro.** Mensagens mudam. Decida pelo `status` (e pelo `type`, se houver).
- **Esperar que o `fetch` lance erro no 404.** Ele só lança quando não há resposta. Confira `resposta.ok`.
- **Esquecer o `await` no `return` dentro do `try`.** A falha acontece depois que a função saiu, e o `catch` não a vê.

Capítulo 9 – Autenticação com JWT

Desde o Capítulo 4, toda requisição ao TaskFlow age como a usuária de teste, graças a uma função provisória. Neste capítulo a API passa a saber, de verdade, quem está pedindo. Você vai entender a diferença entre autenticação e autorização, aprender a guardar senhas com segurança usando o bcrypt, conhecer o JWT, o formato de *token* mais usado em APIs, e criar os *endpoints* de cadastro e de *login*. No final, a função `usuarioAtual` lê o *token* da requisição e, sem que nenhuma rota de tarefas seja alterada, cada pessoa passa a ver só as próprias tarefas.

9.1 Autenticação e autorização

Duas palavras parecidas descrevem duas perguntas diferentes, que toda API protegida precisa responder, nesta ordem:

- **Autenticação**: **quem é você?** A pessoa prova a sua identidade, normalmente com algo que só ela sabe, como a senha. Se a prova falha ou falta, a resposta é `401 Unauthorized`, que, como visto no Capítulo 8, quer dizer "não autenticado".
- **Autorização**: **o que você pode fazer?** Já se sabe quem é a pessoa; falta decidir se ela pode fazer aquela operação naquele recurso. Se não pode, a resposta é `403 Forbidden` ou, quando nem a existência do recurso deve ser revelada, `404`.

No TaskFlow, a autorização já existe desde o Capítulo 4: toda consulta filtra pelo `userId`, e a tarefa de outra pessoa recebe `404`. O que falta é a autenticação. Hoje, o `userId` vem de uma função que sempre devolve a mesma usuária; neste capítulo, ele passa a vir de uma prova de identidade enviada em cada requisição.

Como o HTTP é sem estado (Capítulo 1), o servidor não se lembra de quem fez *login*. A solução usada pelas APIs é dividir o trabalho em dois momentos. No ***login***, a pessoa envia e-mail e senha uma única vez e recebe um *token*, uma espécie de crachá. Nas **demais requisições**, ela mostra o crachá, e não a senha, no cabeçalho `Authorization`:

```
1. POST /api/auth/login  {"email": ..., "password": ...}
   <- 200 {"token": "eyJhbGciOi...", ...}

2. GET /api/tasks
   Authorization: Bearer eyJhbGciOi...
   <- 200 as tarefas de quem é dono do token
```

A palavra `Bearer` significa "portador": quem porta o *token* é tratado como o dono dele. Isso torna o *token* tão valioso quanto a senha enquanto ele vale, e é por isso que ele só deve trafegar por HTTPS e ter prazo de validade, assuntos que voltam ao longo do capítulo.

9.2 Nunca guarde a senha

Antes do *login*, vem o cadastro, e com ele a primeira decisão de segurança: como guardar a senha. A resposta é: **não guarde**. Bancos de dados vazam, por invasão, por um *backup* esquecido ou por um funcionário descuidado; se as senhas estiverem lá, quem obtiver a cópia entra na conta de todas as pessoas, e não só no seu sistema, porque muita gente repete a senha em vários serviços.

O que se guarda é um ***hash*** da senha. Uma função de *hash* transforma um texto em outro, de tamanho fixo, de um jeito que não tem volta: a partir do *hash*, não há como calcular a senha. Para conferir um *login*, o servidor calcula o *hash* da senha digitada e compara com o guardado. Não é criptografia: algo criptografado pode ser decifrado com a chave certa; um *hash*, não.

Mas não serve qualquer função de *hash*. As funções rápidas, como SHA-256, foram feitas para calcular milhões de *hashes* por segundo, e isso ajuda o atacante: com a cópia do banco, ele testa as senhas mais comuns, uma a uma, contra cada *hash*. Para senhas, existem funções feitas para ser **lentas**, como o **bcrypt**, o scrypt e o Argon2. O TaskFlow usa o bcrypt, pela biblioteca `bcryptjs`, instalada no Capítulo 3 para o *seed*. Veja o que ela faz com a mesma senha, duas vezes:

```
import bcrypt from "bcryptjs";

await bcrypt.hash("senha123", 10);
// $2b$10$An2hg.fjDASVHkPrIKgAUuLT3q7EjIrD7MJFtAGnGBLcMMsnJNhh2
await bcrypt.hash("senha123", 10);
// $2b$10$t9ADoUij0L3VeLGHkJJtQ.hiyIgTEMcfmeYA2HtSzejTmCJdo4XbW

const guardado = "$2b$10$An2hg.fjDASVHkPr...";
await bcrypt.compare("senha123", guardado);   // true
await bcrypt.compare("Senha123", guardado);   // false
```

Os dois resultados são diferentes porque o bcrypt sorteia um ***salt***, um valor aleatório que entra no cálculo junto com a senha. Sem o *salt*, duas pessoas com a mesma senha teriam o mesmo *hash*, e um atacante poderia usar tabelas prontas, com os *hashes* de milhões de senhas comuns, calculadas uma única vez. Com o *salt*, cada *hash* precisa ser atacado separadamente. O *hash* do bcrypt guarda tudo o que é preciso para conferi-lo depois:

```
$2b$ 10 $ An2hg.fjDASVHkPrIKgAUu LT3q7EjIrD7MJFtAGnGBLcMMsnJNhh2
 |    |   |                     |
 |    |   salt (22 caracteres)   hash (31 caracteres)
 |    custo
 versão do algoritmo
```

Por isso o `compare` não precisa receber o *salt*: ele o lê do próprio *hash* guardado. O **custo** controla a lentidão: cada ponto a mais dobra o tempo de cálculo. Estes tempos foram medidos com o `bcryptjs` em um computador comum:

Custo	Tempo por *hash*	Senhas testadas por segundo
8	33 ms	cerca de 30
10	125 ms	cerca de 8
12	463 ms	cerca de 2
14	1544 ms	menos de 1

Para quem faz *login*, um décimo de segundo passa despercebido; para quem tenta adivinhar senhas, é a diferença entre testar bilhões e testar algumas por segundo. O TaskFlow usa o custo 10, o mesmo do *seed*; muitos projetos usam 12. O certo é o maior custo que o servidor aguenta sem deixar o *login* lento, revisto de tempos em tempos,

porque os computadores ficam mais rápidos.

Atenção: o limite de 72 bytes

O bcrypt usa só os primeiros 72 bytes da senha e ignora o resto, sem avisar: no teste, um *hash* de 72 letras "a" seguidas de "X" aceitou 72 letras "a" seguidas de "Y". Por isso o cadastro do TaskFlow recusa senhas maiores que 72 bytes. São bytes, e não letras: no UTF-8, uma letra acentuada ocupa 2 bytes, e um emoji, 4. O Argon2, vencedor de uma competição de funções de *hash* de senha em 2015, não tem esse limite e é a recomendação atual do OWASP para projetos novos; o bcrypt continua aceito e é o mais usado.

9.3 Sessão ou token?

Depois do *login*, há duas formas de lembrar quem a pessoa é. Na **sessão**, a forma tradicional dos sites, o servidor cria um registro ("a sessão 8f3a pertence à Bia"), guarda-o no banco ou na memória e entrega ao navegador só o número da sessão, em um *cookie*. A cada requisição, o servidor consulta o registro. No ***token* autocontido**, o servidor não guarda nada: ele entrega um *token* que já diz quem é a pessoa e até quando vale, assinado para que ninguém possa alterá-lo. A cada requisição, basta conferir a assinatura.

	Sessão	*Token* (JWT)
o servidor guarda	a sessão de cada pessoa	nada
conferir	uma consulta ao registro	conferir a assinatura
encerrar antes da hora	apagar o registro	difícil: ele vale até vencer
uso mais comum	sites, com *cookies*	APIs, aplicativos e serviços

O *token* combina com a ideia de API sem estado do Capítulo 2: qualquer servidor que conheça a chave confere o *token*, sem consultar um registro central, e o mesmo *token* serve para um aplicativo de celular, para um *site* e para outro programa. O preço está na terceira linha: um *token* roubado vale até vencer. Por isso, *tokens* de acesso têm vida curta; no TaskFlow, uma hora.

9.4 Por dentro de um JWT

O **JWT** (*JSON Web Token*, RFC 7519, pronuncia-se "jót") é o formato de *token* mais usado. Este é um *token* real, emitido pelo TaskFlow durante os testes deste capítulo (ele é uma linha só; as quebras são da página):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJzdWIiOiJjbXZja3plYjYwMDAwcm92OXpjM3dxb2QxIiwiaWF0Ijox
 NzkyMjUyNzAxLCJleHAiOjE3OTIyNTYzMDF9
.M-WrUp784di_l2karesKVIUyP3qxAu42iFfTv0V5pBE
```

São três partes, separadas por pontos: o **cabeçalho**, a **carga** (*payload*) e a **assinatura**. As duas primeiras são objetos JSON codificados em **Base64URL**, uma forma de escrever bytes só com letras, números, - e _, que podem ir em uma URL ou em um

cabeçalho sem problemas. Base64URL não é criptografia: qualquer pessoa decodifica, sem chave nenhuma.

```
const [cabecalho, carga] = token.split(".");
Buffer.from(cabecalho, "base64url").toString();
// {"alg":"HS256","typ":"JWT"}
Buffer.from(carga, "base64url").toString();
// {"sub":"cmvckzeb60000rov9zc3wqod1",
//  "iat":1792252701,"exp":1792256301}
```

O cabeçalho diz que o *token* é um JWT assinado com o algoritmo HS256. A carga traz as ***claims***, as afirmações do *token*. A RFC 7519 reserva alguns nomes curtos:

Claim	Nome	Conteúdo
`sub`	*subject*	de quem é o *token*; no TaskFlow, o `id` do usuário
`iat`	*issued at*	quando foi emitido
`exp`	*expiration*	quando vence; depois disso, é recusado
`iss`	*issuer*	quem emitiu, como `"taskflow"`
`aud`	*audience*	a quem se destina, como `"taskflow-api"`

As datas são contadas em segundos desde 1/1/1970, em UTC, e não em milissegundos, como no `Date.now()` do JavaScript. O `iat` 1792252701 é 17/10/2026, às 15h58 UTC; o `exp` é exatamente 3600 segundos, uma hora, depois.

A terceira parte é a que dá segurança ao *token*. O servidor junta o cabeçalho e a carga e calcula um HMAC-SHA256 (o "HS256") com uma **chave secreta**, que só ele conhece. Se alguém trocar o `sub` da carga para o de outra pessoa, a assinatura não confere mais, e calcular a assinatura certa exige a chave. Assim, o servidor confia na carga sem guardar nada: se a assinatura confere, foi ele mesmo quem emitiu o *token*, e ninguém o alterou depois.

Atenção: assinado não é secreto

Qualquer pessoa que tenha o *token* lê a carga, como foi feito acima. Nunca coloque nela senhas, dados pessoais ou qualquer coisa que o portador não possa ver. O TaskFlow coloca só o `id`. Também evite dados que mudam, como o nome ou o papel da pessoa: o *token* continuaria afirmando o valor antigo até vencer.

9.5 A chave secreta

Com o HS256, a mesma chave assina e confere os *tokens*. Quem a tiver pode emitir *tokens* em nome de qualquer pessoa, então ela é tratada como a senha do banco: fica no `.env`, nunca no código nem no GitHub. Ela também precisa ser longa e aleatória, e não uma palavra, porque um atacante com um *token* na mão pode testar chaves fora do servidor, à vontade, até encontrar a que reproduz a assinatura. Gere uma com o próprio Node.js:

```
$ node -e "console.log(require('node:crypto')
    .randomBytes(32).toString('base64url'))"
CLCKrqr-0uE-DTNmvKsoi-UFZVM6MncslTufAm217KY
```

O comando é uma linha só. São 32 bytes sorteados, 256 bits, o tamanho que a RFC 7518 exige para o HS256. Acrescente a chave ao `.env`, sem copiar a deste exemplo, que agora é pública:

```
# .env -- NUNCA vai para o Git
DATABASE_URL="postgresql://..."
DIRECT_URL="postgresql://..."
JWT_SECRET="cole-aqui-a-chave-gerada"
```

Trocar a chave invalida, de uma vez, todos os *tokens* emitidos: é o que se faz quando se suspeita de que ela vazou. Cada ambiente, o seu computador e a Vercel (Capítulo 12), deve ter a sua própria chave.

9.6 Criando e conferindo o token

Para criar e conferir os *tokens*, o TaskFlow usa a biblioteca `jose`, que segue as RFCs do JWT, não depende de outros pacotes e funciona tanto no Node.js quanto em ambientes mais restritos, como os servidores de borda (*edge*):

```
$ npm install jose
```

Tudo o que diz respeito aos *tokens* fica em um arquivo:

```
// lib/token.ts -- cria e confere os tokens JWT da API
import { SignJWT, jwtVerify } from "jose";

const texto = process.env.JWT_SECRET;
if (!texto || texto.length < 32) {
  throw new Error("Defina no .env um JWT_SECRET de 32+ letras.");
}
const segredo = new TextEncoder().encode(texto);

const DURACAO = 60 * 60; // 1 hora, em segundos

export async function criarToken(userId: string) {
  const token = await new SignJWT()
    .setProtectedHeader({ alg: "HS256", typ: "JWT" })
    .setSubject(userId)
    .setIssuedAt()
    .setExpirationTime(`${DURACAO}s`)
    .sign(segredo);
  return { token, tokenType: "Bearer", expiresIn: DURACAO };
}

// Devolve o id do usuário (o "sub"), ou null se o token tiver
```

```
// sido alterado, estiver vencido ou não for um JWT
export async function conferirToken(token: string) {
  try {
    const { payload } = await jwtVerify(token, segredo, {
      algorithms: ["HS256"],
    });
    return payload.sub ?? null;
  } catch {
    return null;
  }
}
```

O `if` do começo é uma proteção contra um erro de configuração: sem ele, um `JWT_SECRET` esquecido viraria uma chave vazia, e a API assinaria *tokens* que qualquer um conseguiria falsificar. É melhor a aplicação não subir do que subir insegura. O `TextEncoder` transforma o texto da chave nos bytes que a `jose` espera.

A `criarToken` monta o JWT peça por peça: o cabeçalho, o `sub`, o `iat` (a hora atual) e o `exp`. Ela devolve, além do *token*, o tipo e a validade em segundos, os mesmos nomes de campo que o OAuth 2.0 usa (em *snake_ case*, `token_type` e `expires_in`); assim o cliente sabe quando vai precisar de um novo *token*, sem decodificá-lo.

A `jwtVerify` confere a assinatura e o `exp` e lança um erro se algo estiver errado. A `conferirToken` transforma esse erro em `null`: para o TaskFlow, não importa **por que** o *token* é inválido, só que ele é. A opção `algorithms: ["HS256"]` fecha uma porta conhecida. O algoritmo vem escrito no cabeçalho do próprio *token*, e o padrão JWT prevê até o `"alg": "none"`, "sem assinatura"; algumas bibliotecas antigas obedeciam ao cabeçalho e aceitavam *tokens* sem assinatura nenhuma. Quem decide o algoritmo é o servidor, nunca o *token*. A `jose` já recusa o `none` por padrão, mas deixar a regra escrita documenta a decisão.

9.7 O que o cadastro e o login aceitam

O cadastro e o *login* recebem dados do cliente, então começam, como as tarefas no Capítulo 6, por um esquema do Zod:

```
// lib/esquemas/auth.ts -- o que a API aceita no cadastro e no
// login
import * as z from "zod";

// O e-mail é guardado sem espaços e em minúsculas: assim,
// "Ana@Mail.com" e "ana@mail.com " são a mesma conta
const email = z
  .string({ error: "O e-mail precisa ser um texto." })
  .trim()
  .toLowerCase()
  .pipe(z.email({ error: "Informe um e-mail válido." }));

export const esquemaCadastro = z.object({
  name: z
    .string({ error: "O nome precisa ser um texto." })
```

```
    .trim()
    .min(1, { error: "O nome é obrigatório." })
    .max(100, { error: "O nome pode ter até 100 caracteres." }),
  email,
  password: z
    .string({ error: "A senha precisa ser um texto." })
    .min(8, {
      error: "A senha precisa ter ao menos 8 caracteres.",
    })
    .refine((senha) => Buffer.byteLength(senha) <= 72, {
      error: "A senha é longa demais (máximo de 72 bytes).",
    }),
}, { error: "O corpo deve ser um objeto JSON." });

// No login, a senha não passa pelas regras do cadastro: elas
// podem mudar, e quem já tem conta precisa continuar entrando.
export const esquemaLogin = z.object({
  email,
  password: z
    .string({ error: "A senha precisa ser um texto." })
    .min(1, { error: "A senha é obrigatória." }),
}, { error: "O corpo deve ser um objeto JSON." });

export type Cadastro = z.infer<typeof esquemaCadastro>;
export type Login = z.infer<typeof esquemaLogin>;
```

O e-mail é usado nos dois esquemas, por isso fica em uma constante. O `pipe` encadeia dois esquemas: primeiro o texto é limpo e passado para minúsculas, depois o resultado é conferido como e-mail. Sem essa normalização, "Ana@Mail.com" e "ana@mail.com" seriam duas contas diferentes, e a pessoa que se cadastrou de um jeito não conseguiria entrar digitando do outro.

A senha **não** recebe `trim`: espaços no começo ou no fim fazem parte dela, se a pessoa quis assim. O mínimo de 8 caracteres é o mais comum; o NIST, o órgão de padrões dos Estados Unidos, aceita 8 quando a conta tem um segundo fator de autenticação, como um código no celular, e recomenda ao menos 15 quando a senha é a única proteção. O máximo é o dos 72 bytes do bcrypt, conferido com o `Buffer.byteLength`, que conta bytes, e não letras. O NIST também desaconselha as regras de composição, como "uma maiúscula, um número e um símbolo": elas levam a senhas previsíveis, como `Senha@123`; vale mais uma senha longa.

O campo `name` do cadastro segue as regras do título de uma tarefa. Como no Capítulo 6, o `z.object` descarta os campos que não estão no esquema: um cliente que envie `"id"` ou `"createdAt"` no cadastro não consegue escolhê-los.

9.8 O serviço de usuários

As operações com o banco ficam na camada de serviço, como as das tarefas no Capítulo 7:

```
// lib/servicos/usuarios.ts -- cadastro e conferência de senha
import bcrypt from "bcryptjs";
import { db } from "@/lib/db";
import { Prisma } from "@/app/generated/prisma/client";
import type { Cadastro, Login } from "@/lib/esquemas/auth";

const CUSTO = 10; // o mesmo do seed

// Os campos públicos de um usuário: a senha nunca sai daqui
const camposUsuario = {
  id: true,
  name: true,
  email: true,
  createdAt: true,
} satisfies Prisma.UserSelect;

// Devolve o usuário criado, ou null se o e-mail já existir
export async function cadastrarUsuario(dados: Cadastro) {
  const hash = await bcrypt.hash(dados.password, CUSTO);
  try {
    return await db.user.create({
      data: { ...dados, password: hash },
      select: camposUsuario,
    });
  } catch (erro) {
    if (
      erro instanceof Prisma.PrismaClientKnownRequestError &&
      erro.code === "P2002"
    ) {
      return null;
    }
    throw erro;
  }
}
```

O `camposUsuario` faz para o usuário o que o `camposTarefa` do Capítulo 5 faz para a tarefa: define os campos públicos. A senha, mesmo em forma de *hash*, nunca sai do banco por esta função.

Para descobrir se o e-mail já existe, seria natural buscar o usuário antes de criar. Mas entre a busca e a criação há um intervalo, e duas requisições de cadastro com o mesmo e-mail, chegando ao mesmo tempo, passariam as duas pela busca. Quem garante que o e-mail é único é o banco, pelo `@unique` do `schema.prisma`: a segunda inserção falha com o código `P2002`, "violação de restrição única". A função tenta criar e trata essa falha, a mesma técnica usada com o `P2025` no Capítulo 7. Agora, o *login*:

```
// Um hash de uma senha que ninguém tem (veja o texto abaixo)
const HASH_FALSO = bcrypt.hashSync("ninguem-tem-esta", CUSTO);

// Devolve o id do usuário se e-mail e senha conferem, ou null
export async function autenticar(dados: Login) {
```

```
  const usuario = await db.user.findUnique({
    where: { email: dados.email },
    select: { id: true, password: true },
  });
  const confere = await bcrypt.compare(
    dados.password,
    usuario?.password ?? HASH_FALSO,
  );
  return usuario && confere ? usuario.id : null;
}
```

Aqui, sim, a senha precisa sair do banco, para ser conferida, e por isso o `select` a pede. Ela não sai da função: o resultado é só o `id`, ou `null`.

O `HASH_FALSO` parece estranho, mas resolve um problema real. Seria mais simples devolver `null` logo que o e-mail não fosse encontrado. Só que o `compare` leva um décimo de segundo, e a busca no banco, poucos milissegundos. Essa versão mais simples foi medida no TaskFlow, com 10 tentativas de *login* com a senha errada para cada caso: com um e-mail cadastrado, 334 ms em média; com um e-mail que não existe, 35 ms. Pelo tempo de resposta, qualquer pessoa descobriria quem tem conta, mesmo que a mensagem de erro fosse a mesma. Esse tipo de vazamento se chama **ataque de temporização** (*timing attack*). Com o `HASH_FALSO`, o `compare` roda nos dois casos, e as médias ficaram em 291 ms e 272 ms, indistinguíveis.

Atenção: o cadastro também revela contas

A mensagem "e-mail ou senha incorretos" e o `HASH_FALSO` escondem do *login* quem tem conta, mas o cadastro responde `409` quando o e-mail já existe, e isso também revela. Esconder de verdade exige mudar o fluxo: o cadastro sempre responde "enviamos um e-mail de confirmação", e é o e-mail que diz à pessoa que ela já tem conta. O TaskFlow fica com o `409`, combinado no Capítulo 2, que é mais simples e é o que a maioria das APIs faz. Contra quem tenta milhares de e-mails ou senhas, a defesa é limitar as tentativas, com o `429 Too Many Requests` (Capítulo 12).

9.9 As rotas de cadastro e login

Com o esquema e o serviço prontos, as rotas ficam curtas. Elas seguem o padrão do Capítulo 8: envolvidas pela `comErros`, com a validação lançando o `400` e os outros erros devolvidos com `return problema(...)`.

```
// app/api/auth/register/route.ts -- cria uma conta
import type { NextRequest } from "next/server";
import { comErros, problema } from "@/lib/erros";
import { validar } from "@/lib/validar";
import { esquemaCadastro } from "@/lib/esquemas/auth";
import { cadastrarUsuario } from "@/lib/servicos/usuarios";

// POST /api/auth/register
export const POST = comErros(async (request: NextRequest) => {
  const corpo = await request.json().catch(() => null);
```

```
  const dados = validar(esquemaCadastro, corpo);

  const usuario = await cadastrarUsuario(dados);
  if (!usuario) {
    return problema(409, "Este e-mail já está em uso.");
  }
  return Response.json(usuario, { status: 201 });
});
```

O cadastro responde `201` com o usuário, sem a senha, como combinado no Capítulo 2. Diferente da criação de tarefas, não há cabeçalho `Location`, porque a API não tem um endereço como `/api/users/:id` para consultar a conta; o cadastro também não faz *login* automático, e a pessoa entra em seguida, pela rota de *login*:

```
// app/api/auth/login/route.ts -- troca e-mail e senha por um
// token
import type { NextRequest } from "next/server";
import { comErros, problema } from "@/lib/erros";
import { validar } from "@/lib/validar";
import { esquemaLogin } from "@/lib/esquemas/auth";
import { autenticar } from "@/lib/servicos/usuarios";
import { criarToken } from "@/lib/token";

// POST /api/auth/login
export const POST = comErros(async (request: NextRequest) => {
  const corpo = await request.json().catch(() => null);
  const dados = validar(esquemaLogin, corpo);

  const userId = await autenticar(dados);
  if (!userId) {
    return problema(401, "E-mail ou senha incorretos.");
  }
  // um token não deve ficar guardado em caches pelo caminho
  return Response.json(await criarToken(userId), {
    headers: { "Cache-Control": "no-store" },
  });
});
```

A mensagem de erro é a mesma para o e-mail que não existe e para a senha errada; dizer qual dos dois falhou entregaria, de novo, quem tem conta. O `login` é uma operação, e não a leitura de um recurso, por isso usa `POST`; e nunca `GET`, porque a senha iria na URL e ficaria registrada no histórico do navegador e nos *logs* dos servidores. O cabeçalho `Cache-Control: no-store`, exigido pelo OAuth 2.0 nas respostas com *token*, proíbe que a resposta seja guardada por qualquer *cache*, no navegador ou no caminho.

9.10 O usuário atual, agora de verdade

Chegou a hora de aposentar a função provisória do Capítulo 4. A nova `usuarioAtual` lê o cabeçalho `Authorization`, confere o *token* e devolve o usuário, com os mesmos campos de antes, `id` e `name`. Se algo falhar, ela lança o `401`, como o Capítulo 8 prometeu:

```
// lib/usuario-atual.ts -- quem está fazendo a requisição,
// descoberto pelo token JWT do cabeçalho Authorization
import { headers } from "next/headers";
import { db } from "@/lib/db";
import { ErroApi } from "@/lib/erros";
import { conferirToken } from "@/lib/token";

export async function usuarioAtual() {
  // "Authorization: Bearer eyJhbGciOi..."
  const autorizacao = (await headers()).get("authorization");
  const [tipo, token] = autorizacao?.split(" ") ?? [];
  if (tipo?.toLowerCase() !== "bearer" || !token) {
    throw new ErroApi(401, "Envie o token no cabeçalho "
      + "Authorization: Bearer <token>.");
  }

  const userId = await conferirToken(token);
  if (!userId) {
    throw new ErroApi(401, "Token inválido ou expirado.");
  }

  // a conta pode ter sido excluída depois que o token foi criado
  const usuario = await db.user.findUnique({
    where: { id: userId },
    select: { id: true, name: true },
  });
  if (!usuario) {
    throw new ErroApi(401, "Token inválido ou expirado.");
  }
  return usuario;
}
```

A função não recebe a requisição, e mesmo assim lê os cabeçalhos dela. Quem permite isso é a `headers()`, do pacote `next/headers`: chamada dentro de uma rota, ela devolve os cabeçalhos da requisição que está sendo atendida. É o que mantém a promessa do Capítulo 4: as rotas continuam chamando `usuarioAtual()`, sem argumentos, e nenhuma delas precisa mudar.

O `split(" ")` separa `"Bearer eyJ..."` em duas partes. O nome do esquema é comparado em minúsculas, porque o HTTP não diferencia maiúsculas de minúsculas nele: `bearer` também vale. A busca no banco, no fim, tem um motivo: o *token* continua válido por até uma hora, mesmo que a conta seja excluída nesse meio-tempo, e só o banco sabe disso. A versão provisória já fazia uma consulta por requisição, então o custo não mudou.

As três falhas recebem o `401`, mas com mensagens diferentes para dois casos: a falta do *token*, que é um erro de quem programa o cliente, e o *token* recusado, que normalmente só pede um novo *login*. Não há como saber por que o *token* foi recusado, e isso é proposital: vencido, adulterado ou de uma conta excluída, a resposta é a mesma.

Falta um detalhe do `401`. A RFC 9110 diz que essa resposta deve trazer o cabeçalho `WWW-Authenticate`, que informa ao cliente qual esquema de autenticação usar. Como todos os erros passam pela `problema` do Capítulo 8, basta acrescentá-lo lá, para todo

401:

```
// Monta a resposta de erro; os extras (errors, instance) são
// acrescentados ao corpo quando existem
export function problema(
  status: number,
  detail: string,
  extras: { errors?: Campos; instance?: string } = {},
) {
  const headers = new Headers({
    "Content-Type": "application/problem+json",
  });
  // RFC 9110: o 401 diz qual esquema de autenticação usar
  if (status === 401) headers.set("WWW-Authenticate", "Bearer");

  return Response.json(
    {
      title: titulos[status] ?? "Erro",
      status,
      detail,
      ...extras,
    },
    { status, headers },
  );
}
```

O `Headers` é a classe padrão do `fetch` para um conjunto de cabeçalhos; com ela, um cabeçalho pode ser acrescentado depois de o objeto ser criado, com o `set`. O resto da função não mudou.

Repare também na ordem das verificações combinada no Capítulo 8: a autenticação vem antes da validação. Como as rotas de tarefas chamam `usuarioAtual()` na primeira linha, quem não tem *token* recebe `401` mesmo enviando dados inválidos, e não aprende nada sobre as regras da API. O projeto fica assim:

```
app/api/
  [...rota]/route.ts
  auth/register/route.ts  POST (cadastro)
  auth/login/route.ts     POST (login, devolve o token)
  health/route.ts
  tasks/route.ts          sem mudanças
  tasks/[id]/route.ts     sem mudanças
lib/
  erros.ts                o 401 com WWW-Authenticate
  esquemas/auth.ts        cadastro e login
  servicos/usuarios.ts    cadastrarUsuario, autenticar
  token.ts                criarToken, conferirToken
  usuario-atual.ts        agora lê o token
```

9.11 Testando a autenticação

Estes são resultados reais da API com o código deste capítulo. Todas as respostas de erro trazem `Content-Type: application/problem+json`, e todas as respostas `401`, também `WWW-Authenticate: Bearer`. Primeiro, o cadastro e o *login*:

```
POST /api/auth/register
     {"name":"Bia Souza","email":" Bia@Mail.com ",
      "password":"minhasenha"}
201 {"id":"cmvckzeb60000rov9zc3wqod1","name":"Bia Souza",
     "email":"bia@mail.com",
     "createdAt":"2026-10-17T15:58:20.466Z"}

POST /api/auth/register  (o mesmo e-mail de novo)
409 {"title":"Conflito","status":409,
     "detail":"Este e-mail já está em uso."}

POST /api/auth/register
     {"name":"","email":"bia","password":"123"}
400 {"title":"Requisição inválida","status":400,
     "detail":"Um ou mais campos estão inválidos.",
     "errors":{"name":["O nome é obrigatório."],
               "email":["Informe um e-mail válido."],
               "password":["A senha precisa ter ao menos
                            8 caracteres."]}}

POST /api/auth/login
     {"email":"bia@mail.com","password":"errada123"}
401 {"title":"Não autenticado","status":401,
     "detail":"E-mail ou senha incorretos."}

POST /api/auth/login
     {"email":"BIA@mail.com","password":"minhasenha"}
200 Cache-Control: no-store
    {"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
     "tokenType":"Bearer","expiresIn":3600}
```

O e-mail foi guardado limpo e em minúsculas, e o *login* com `BIA@mail.com` funcionou. A mesma resposta `401` veio para a senha errada e para um e-mail inexistente. Agora, as tarefas:

```
GET /api/tasks           (sem o cabeçalho Authorization)
401 {"title":"Não autenticado","status":401,
     "detail":"Envie o token no cabeçalho Authorization:
               Bearer <token>."}

GET /api/tasks           (com o token da Bia)
200 {"data":[],"meta":{"page":1,"limit":20,"total":0,
     "totalPages":0}}

POST /api/tasks  {"title":"Tarefa da Bia"}   (token da Bia)
201 a tarefa, com Location: /api/tasks/cmvckzfnv0002...
```

```
GET /api/tasks?limit=2   (com o token da demo)
200 as tarefas do seed; meta: {"total":5, ...}

GET /api/tasks/cmvckzfnv0002...   (a tarefa da Bia, token da demo)
404 {"title":"Não encontrado","status":404,
     "detail":"Tarefa não encontrada."}

GET /api/tasks?limit=500  e  POST /api/tasks {"title":""}
(sem token)
401 ... "Envie o token no cabeçalho Authorization: ..."
```

A Bia começou sem nenhuma tarefa, as cinco do *seed* continuam sendo só da demo, e cada uma recebe `404` na tarefa da outra. Os dois últimos pedidos, com dados inválidos e sem *token*, receberam `401`, e não `400`. Por fim, os *tokens* que não deveriam passar:

```
Authorization: Bearer abc.def.ghi           (não é um JWT)
Token da Bia, com o "sub" trocado pelo da demo
Token com {"alg":"none"} e sem assinatura
Token vencido (o "exp" de uma hora atrás)
Token assinado com outra chave
401 {"title":"Não autenticado","status":401,
     "detail":"Token inválido ou expirado."}

Authorization: bearer eyJ...   (com b minúsculo)
200 as tarefas
```

Os cinco *tokens* falsos receberam a mesma resposta. O teste mais importante é o segundo: trocar o `sub` é exatamente o que um atacante tentaria para ver as tarefas de outra pessoa, e a assinatura o impediu.

9.12 Do lado do cliente

No REST Client do Capítulo 4, uma requisição pode receber um nome, e as seguintes podem usar campos da resposta dela. Assim, o *token* não precisa ser copiado à mão:

```
### Entrar com a usuária de teste do seed
# @name login
POST http://localhost:3000/api/auth/login
Content-Type: application/json

{"email": "demo@taskflow.com", "password": "senha123"}

### Listar as tarefas, com o token do login acima
GET http://localhost:3000/api/tasks
Authorization: Bearer {{login.response.body.token}}
```

Rode primeiro o *login*; enquanto o *token* valer, as outras requisições do arquivo o usam. Em um programa, a ideia é a mesma: guardar o *token* recebido e enviá-lo em todas as chamadas. Uma função que faz isso, e trata o `401`:

```
let token = null;

async function entrar(email, password) {
  const resposta = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!resposta.ok) return false;
  token = (await resposta.json()).token;
  return true;
}

async function api(caminho, opcoes = {}) {
  const resposta = await fetch(caminho, {
    ...opcoes,
    headers: {
      ...opcoes.headers,
      Authorization: `Bearer ${token}`,
    },
  });
  if (resposta.status === 401) {
    token = null;   // vencido ou inválido: pedir novo login
  }
  return resposta;
}
```

A pergunta seguinte é **onde guardar** o *token*. Em um aplicativo de celular, no armazenamento seguro do sistema (Keychain, no iOS, e Keystore, no Android). Em um navegador, não há resposta perfeita. No `localStorage`, ele sobrevive a recarregar a página, mas qualquer *script* da página consegue lê-lo, inclusive um *script* malicioso injetado por uma falha de XSS. Em uma variável, como acima, o risco é menor, mas o *token* se perde ao recarregar a página. Muitos sistemas web preferem não expor o *token* ao JavaScript e o guardam em um *cookie* `HttpOnly`, que os *scripts* não leem; essa escolha traz outros cuidados, como a proteção contra CSRF, que voltam no Capítulo 12.

E o *logout*? Com *tokens* autocontidos, o servidor não tem o que apagar: o *logout* é o cliente descartar o *token*. Um *token* copiado antes disso continua valendo até o `exp`, e é por isso que a validade é curta. Sistemas que precisam de sessões longas costumam combinar um *token* de acesso curto, como o do TaskFlow, com um ***refresh token*** de longa duração, guardado no banco, que serve só para pedir novos *tokens* de acesso e pode ser revogado a qualquer momento. Esse é o modelo do OAuth 2.0, o protocolo por trás dos botões "Entrar com Google".

Erros Comuns

- **Guardar a senha, ou um *hash* rápido dela.** Use uma função feita para senhas, como bcrypt ou Argon2, com *salt* e custo adequado. MD5 e SHA-256 não servem.
- **Colocar dados sensíveis na carga do JWT.** Ela é só codificada, e qualquer pessoa a lê. Guarde nela só o identificador.

- **Uma chave secreta fraca ou no código.** Com a chave, qualquer um emite *tokens*. Gere 32 bytes aleatórios e guarde-os no `.env`.
- **Aceitar o algoritmo que o *token* informa.** Fixe o algoritmo no servidor, com a opção `algorithms`.
- **Usar `decodeJwt` no lugar de `jwtVerify`.** Decodificar só lê a carga; quem confere a assinatura e a validade é o `jwtVerify`.
- ***Token* sem validade.** Um *token* roubado valeria para sempre. Defina sempre o `exp`, e curto.
- **Mensagens de *login* diferentes para e-mail e senha.** "E-mail não cadastrado" revela quem tem conta. Use uma única mensagem, e o mesmo tempo de resposta.
- **Conferir o e-mail repetido antes de criar a conta.** Duas requisições simultâneas passam pela conferência. Deixe o `@unique` do banco decidir e trate o `P2002`.
- **Enviar a senha por `GET` ou sem HTTPS.** Ela fica na URL, nos *logs* e no histórico, ou trafega aberta. *Login* é `POST`, e sempre por HTTPS.
- **Confundir 401 e 403.** *Token* ausente, vencido ou inválido: `401`. *Token* válido, mas sem permissão: `403`.